

libDwm

binary serialization and deserialization in C++

Version 1.0.0

Contents

1	Introduction	12
1.1	Motivation	12
1.2	History	12
1.3	Primary I/O classes	12
1.4	Directly supported types	12
1.5	Supported std class templates	12
1.6	Concepts	12
2	Design	13
2.1	A primary I/O class for each source/sink type	13
2.2	Conceptual interfaces	13
2.3	Categories of interface overloads in actual implementation	14
2.3.1	Regular functions for directly supported types	14
2.3.2	Function templates for STL containers and other standard class templates	14
2.3.3	Function templates for types that meet the conceptual requirements of a given primary I/O class	15
2.4	Dwm::StreamIO read/write functions	15
2.4.1	Dwm::StreamIO conceptual read/write functions	15
2.5	Dwm::FileIO read/write functions	16
2.5.1	Dwm::FileIO conceptual read/write functions	16
2.6	Dwm::DescriptorIO read/write functions	16
2.6.1	Dwm::DescriptorIO conceptual read/write functions	16
2.7	Dwm::BZ2IO read/write functions	16
2.7.1	Dwm::BZ2IO conceptual read/write functions	16
2.8	Dwm::GZIO read/write functions	17
2.8.1	Dwm::GZIO conceptual read/write functions	17
2.9	Member function overloads	17
2.10	Member function templates	17

<i>CONTENTS</i>	3
2.11 Concepts	17
2.12 Support for user-designed types	17
2.12.1 Before C++26	17
2.12.2 C++26 and beyond	17
3 Getting Started	18
3.1 Write and read a string to/from an iostream	18
3.2 Write and read a map<string,string> to/from an iostream	19
3.3 Turtles all the way down	20
3.4 Integrating a user-defined type	21
3.4.1 With C++26 reflection	21
3.4.2 Without C++26 reflection	21
3.4.3 A test program to write and read std::vector<MyType>	22
3.4.4 Skipping data when using C++26 reflection	23
4 Tips	24
4.1 Implement iostream support first	24
4.2 Shortcuts to supporting other sinks/sources	24
4.3 Resource considerations	24
5 Primary I/O Classes	25
5.1 Dwm::StreamIO	26
5.1.1 Using Dwm::StreamIO with user defined types	26
5.2 Dwm::FileIO	27
5.2.1 Using Dwm::FileIO with user defined types	27
5.3 Dwm::DescriptorIO	28
5.3.1 Using Dwm::DescriptorIO with user defined types	28
5.4 Dwm::BZ2IO	29
5.4.1 Using Dwm::BZ2IO with user defined types	29
5.5 Dwm::GZIO	30
5.5.1 Using Dwm::GZIO with user defined types	30
5.6 Dwm::ASIO	30

6	Other I/O classes, functions and templates	31
6.1	Dwm::EncodedUnsigned<T>	31
6.1.1	Dwm::EncodedU64	31
6.1.2	Dwm::EncodedU32	31
6.1.3	Dwm::EncodedU16	31
6.2	Dwm::EncodedSigned<T>	32
6.2.1	Dwm::EncodedS64	32
6.2.2	Dwm::EncodedS32	32
6.2.3	Dwm::EncodedS16	32
6.3	Dwm::IOUtils	33
6.4	Endian conversions	33
6.4.1	template <typename T> concept IsEndianSensitiveInteger	33
6.4.2	BE2Host	33
6.4.3	Host2BE	33
6.4.4	Bswap	34
7	Directly supported data types	35
7.1	integral types	35
7.2	floating point types	35
7.3	enumerated types	35
7.4	std::string	35
7.5	std::vector<bool>	36
8	Supported std type templates	37
8.1	std::array<class T, std::size_t N>	37
8.2	std::atomic<class T>	37
8.3	std::deque<class T>	37
8.4	std::list<class T>	38
8.5	std::map<KeyType, MappedType>	38
8.6	std::multimap<KeyType, MappedType>	38
8.7	std::optional<class T>	39

8.8	<code>std::pair<class FirstType, class SecondType></code>	39
8.9	<code>std::set<class T></code>	39
8.10	<code>std::multiset<class T></code>	39
8.11	<code>std::unique_ptr<class T></code>	40
8.12	<code>std::unordered_map<class KeyType, class MappedType></code>	40
8.13	<code>std::unordered_multimap<class KeyType, class MappedType></code>	40
8.14	<code>std::unordered_set<class T></code>	40
8.15	<code>std::unordered_multiset<class T></code>	41
8.16	<code>std::vector<class T></code>	41
9	Concepts	42
9.1	Streamable	42
9.2	Concepts used by <code>Dwm::StreamIO</code> (C++ iostreams)	42
9.2.1	<code>HasStreamRead<T></code>	42
9.2.2	<code>HasStreamWrite<T></code>	42
9.2.3	<code>Dwm::IsStreamReadable<T></code>	42
9.2.4	<code>Dwm::IsStreamWritable<T></code>	42
9.3	Concepts used by <code>Dwm::FileIO</code> (FILES, i.e. <code>cstdio</code>)	43
9.3.1	<code>HasFileRead<T></code>	43
9.3.2	<code>HasFileWrite<T></code>	43
9.3.3	<code>Dwm::IsFileReadable<T></code>	43
9.3.4	<code>Dwm::IsFileWritable<T></code>	43
9.4	Concepts used by <code>Dwm::DescriptorIO</code> (UNIX descriptors)	43
9.4.1	<code>HasDescriptorRead<T></code>	43
9.4.2	<code>HasDescriptorWrite<T></code>	43
9.4.3	<code>Dwm::IsDescriptorReadable<T></code>	43
9.4.4	<code>Dwm::IsDescriptorWritable<T></code>	43
9.5	Boost asio sockets	44
9.5.1	<code>HasAsioRead<T></code>	44
9.5.2	<code>HasAsioWrite<T></code>	44

9.6	Concepts used by <code>Dwm::BZ2IO</code> (BZFILES from <code>BZ2_bzopen()</code>)	44
9.6.1	<code>HasBZRead<T></code>	44
9.6.2	<code>HasBZWrite<T></code>	44
9.6.3	<code>Dwm::IsBZ2Readable<T></code>	44
9.6.4	<code>Dwm::IsBZ2Writable<T></code>	44
9.7	Concepts used by <code>Dwm::GZI0</code> (gzFiles from <code>gzopen()</code>)	44
9.7.1	<code>HasGZRead<T></code>	44
9.7.2	<code>HasGZWrite<T></code>	44
9.7.3	<code>Dwm::IsGZReadable<T></code>	44
9.7.4	<code>Dwm::IsGZWritable<T></code>	44
9.8	<code>HasSkipAnnotation<std::meta::info info></code>	44
9.9	Streamed length	45
9.9.1	Constant streamed length	45
9.9.2	Constant streamed length via static <code>constexpr uint64_t StreamedLength()</code> member.	45
9.9.3	Constant streamed length from C++26 reflection	45
10	On-the-wire formats	46
10.1	On-the-wire formats, network byte order (MSB first)	46
10.1.1	16 bit integral type (<code>int16_t</code> , <code>uint16_t</code>) encoding	46
10.1.2	32 bit integral type (<code>int32_t</code> , <code>uint32_t</code>) encoding	46
10.1.3	64 bit integral type (<code>int64_t</code> , <code>uint64_t</code>) encoding	46
10.1.4	float encoding.	46
10.1.5	double encoding	46
10.2	Encoded integral classes	47
10.2.1	<code>Dwm::EncodedU64</code> encoding	47
10.2.2	<code>Dwm::EncodedU32</code> encoding	50
10.2.3	<code>Dwm::EncodedU16</code> encoding	52
10.2.4	<code>Dwm::EncodedS64</code> encoding	54
10.2.5	<code>Dwm::EncodedS32</code> encoding	57
10.2.6	<code>Dwm::EncodedS16</code> encoding	60

10.3 Encoding of std classes and class templates.	62
10.3.1 <code>std::string</code> encoding	62
10.3.2 <code>std::optional<T></code> encoding	62
10.3.3 <code>std::string_view</code> encoding	62
10.3.4 <code>std::optional<T></code> encoding	62
10.3.5 <code>std::pair<FirstType,SecondType></code> encoding	62
10.3.6 <code>std::vector<T></code> encoding	63
10.3.7 <code>std::deque<T></code> encoding	63
10.3.8 <code>std::list<T></code> encoding	63
10.3.9 <code>std::map<Key,Value></code> encoding	63
10.3.10 <code>std::multimap<Key,Value></code> encoding	64
10.3.11 <code>std::unordered_map<Key,Value></code> encoding	64
10.3.12 <code>std::unordered_multimap<Key,Value></code> encoding	64
10.3.13 <code>std::set<T></code> encoding	64
10.3.14 <code>std::multiset<T></code> encoding	64
10.3.15 <code>std::unordered_set<T></code> encoding	65
10.3.16 <code>std::unordered_multiset<T></code> encoding	65

List of Figures

10.1 Dwm::EncodedU64 encoding	47
10.2 Dwm::EncodedU64 example: value 0xAA, big-endian.	48
10.3 Dwm::EncodedU64 example: value 0xAA, little-endian.	48
10.4 Dwm::EncodedU64 example: value 0xAABB, big-endian.	48
10.5 Dwm::EncodedU64 example: value 0xAABB, little-endian.	48
10.6 Dwm::EncodedU64 example: value 0xAABBC, big-endian.	48
10.7 Dwm::EncodedU64 example: value 0xAABBC, little-endian.	49
10.8 Dwm::EncodedU64 example: value 0xAABBCDD, big-endian.	49
10.9 Dwm::EncodedU64 example: value 0xAABBCDD, little-endian.	49
10.10Dwm::EncodedU64 example: value 0xAABBCDDEE, big-endian.	49
10.11Dwm::EncodedU64 example: value 0xAABBCDDEE, little-endian.	49
10.12Dwm::EncodedU64 example: value 0xAABBCDDEEFF, big-endian.	49
10.13Dwm::EncodedU64 example: value 0xAABBCDDEEFF, little-endian.	50
10.14Dwm::EncodedU64 example: value 0xAABBCDDEEFF99, big-endian.	50
10.15Dwm::EncodedU64 example: value 0xAABBCDDEEFF99, little-endian.	50
10.16Dwm::EncodedU64 example: value 0xAABBCDDEEFF9988, big-endian.	50
10.17Dwm::EncodedU64 example: value 0xAABBCDDEEFF9988, little-endian.	50
10.18Dwm::EncodedU32 encoding	51
10.19one byte big-endian value 255	51
10.20one byte little-endian value 255	52
10.21two byte big-endian value 65,518	52
10.22three byte big-endian value 16,772,829	52
10.23four byte big-endian value 86,554,432	52
10.24four byte little-endian value 86,554,432	52
10.25Dwm::EncodedU16 encoding	53
10.26one byte big-endian value 255	53

10.27one byte little-endian value 255	54
10.28two byte big-endian value 65,518	54
10.29Dwm::EncodedS64 encoding	54
10.30one byte big-endian value -255	55
10.31one byte little-endian value -255	55
10.32two byte big-endian value -65,518	55
10.33three byte big-endian value -16,772,829	56
10.34four byte big-endian value -4,293,844,428	56
10.35four byte little-endian value -4,293,844,428	56
10.36five byte big-endian value -1,099,224,173,755	56
10.37five byte little-endian value -1,099,224,173,755	56
10.38six byte big-endian value -281,401,388,481,450	56
10.39six byte little-endian value -281,401,388,481,450	57
10.40seven byte big-endian value -72,038,755,451,251,353	57
10.41seven byte little-endian value -72,038,755,451,251,353	57
10.42eight byte big-endian value -18,441,921,395,520,346,504	57
10.43eight byte little-endian value -18,441,921,395,520,346,504	57
10.44Dwm::EncodedS32 encoding	58
10.45one byte big-endian value -255	59
10.46one byte little-endian value -255	59
10.47two byte big-endian value -65,518	59
10.48three byte big-endian value -16,772,829	59
10.49four byte big-endian value -86,554,432	59
10.50four byte little-endian value -86,554,432	60
10.51Dwm::EncodedS16 encoding	60
10.52one byte big-endian value -255	61
10.53one byte little-endian value -255	61
10.54two byte big-endian value -32,750	61
10.55two byte little-endian value -32,750	61
10.56std::string encoding	62

10.57std::string_view encoding	62
10.58std::pair<FirstType,SecondType> encoding	62
10.59std::vector<T> encoding	63
10.60std::deque<T> encoding	63
10.61std::list<T> encoding	63
10.62std::map<Key,Value> encoding	63
10.63std::multimap<Key,Value> encoding	64
10.64std::unordered_map<Key,Value> encoding	64
10.65std::unordered_multimap<Key,Value> encoding	64
10.66std::set<T> encoding	64
10.67std::multiset<T> encoding	64
10.68std::unordered_set<T> encoding	65
10.69std::unordered_multiset<T> encoding	65

List of Tables

2.1	Primary I/O class source and sink types	13
2.2	Conceptual read/write functions in all primary I/O classes	13
2.3	Conceptual interface mapping to the primary I/O classes	14
2.4	Primary I/O class concepts for user-defined types	15
2.5	Conceptual read and write member functions in <code>Dwm::StreamIO</code>	15
10.1	Meaning of endian bit in <code>Dwm::EncodedU64</code> encoding byte	47
10.2	meaning of size bits in <code>Dwm::EncodedU64</code> encoding byte	47
10.3	Meaning of endian bit in <code>Dwm::EncodedU32</code> encoding byte	51
10.4	meaning of size bits in <code>Dwm::EncodedU32</code> encoding byte	51
10.5	Meaning of endian bit in <code>Dwm::EncodedU16</code> encoding byte	53
10.6	meaning of size bits in encoding byte	53
10.7	Meaning of endian bit in <code>Dwm::EncodedS64</code> encoding byte	54
10.8	Meaning of sign bit in <code>Dwm::EncodedS64</code> encoding byte	55
10.9	meaning of size bits in <code>Dwm::EncodedS64</code> encoding byte	55
10.10	Meaning of endian bit in <code>Dwm::EncodedS32</code> encoding byte	58
10.11	Meaning of sign bit in <code>Dwm::EncodedS32</code> encoding byte	58
10.12	meaning of size bits in <code>Dwm::EncodedS32</code> encoding byte	58
10.13	Meaning of endian bit in <code>Dwm::EncodedS16</code> encoding byte	60
10.14	Meaning of sign bit in <code>Dwm::EncodedS16</code> encoding byte	60
10.15	meaning of size bits in <code>Dwm::EncodedS16</code> encoding byte	61

Chapter 1

Introduction

1.1 Motivation

Efficiency I did not want conversions between textual and native representations for arithmetic types. For example, if I need to stream an unsigned 32-bit integer of value 0xFFFFFFFF, I should only need to transfer 4 bytes, not 10 or more.

Portability Since network transport was a primary goal, I was motivated to provide schemes that work across fairly common hardware platforms and UNIX operating systems.

Flexibility I/O facilities should be flexible enough to accomodate instances of common class templates from the standard library, particularly containers like `vector`, `map`, et. al. They should also be flexible enough to accomodate user-defined types without a tremendous amount of work.

1.2 History

libDwm was started around 1998. Needless to say, C++ was a very different language in 1998 versus 2025. And the standard library has grown tremendously since 1998.

The I/O facilities in libDwm started appearing in the early 2000's (around the time we got C++03). Since that time, it has evolved a bit with each new version of the C++ standard, and is about to see a sea change with C++26 reflection (P2996 and P3394 in particular).

1.3 Primary I/O classes

1.4 Directly supported types

1.5 Supported std class templates

1.6 Concepts

Chapter 2

Design

2.1 A primary I/O class for each source/sink type

The library divides I/O functionality into a primary I/O class for a given type of source and sink. For example, `Dwm::StreamIO` provides functions for `std::istream` sources and `std::ostream` sinks. Table 2.1 lists the primary I/O classes and their associated source and sink type.

Primary I/O class	Source type	Sink type	Description
<code>Dwm::StreamIO</code>	<code>std::istream &</code>	<code>std::ostream &</code>	references to C++ iostreams
<code>Dwm::FileIO</code>	<code>FILE *</code>	<code>FILE *</code>	FILE created with <code>fopen()</code> and friends
<code>Dwm::DescriptorIO</code>	<code>int</code>	<code>int</code>	UNIX descriptors
<code>Dwm::BZ2IO</code>	<code>BZFILE *</code>	<code>BZFILE *</code>	BZFILE created with <code>BZ2_bzopen()</code>
<code>Dwm::GZIO</code>	<code>gzFile</code>	<code>gzFile</code>	gzFile created with <code>gzopen()</code>

Table 2.1: Primary I/O class source and sink types

Each of the primary I/O classes provides a set of static functions and function templates to read and write data.

2.2 Conceptual interfaces

Conceptually, the I/O classes provide four primary interfaces to read and write from/to their associated source/sink type, as depicted in table 2.2. There are single-object interfaces (`ReadFunc` and `WriteFunc`) and multi-object variadic interfaces (`VariadicReadFunc` and `VariadicWriteFunc`). The multi-object variadic interfaces just call to the single-object interfaces for each object.

Read/write functions (conceptual)
<pre>template <typename T> static <i>ReadReturnType</i> & <i>ReadFunc</i>(<i>SourceType</i> source, T & t); template <typename T> static <i>WriteReturnType</i> <i>WriteFunc</i>(<i>SinkType</i> sink, const T & t); template <typename... Args> static <i>ReadReturnType</i> <i>VariadicReadFunc</i>(<i>SourceType</i> source, Args & ...args); template <typename... Args> static <i>WriteReturnType</i> <i>VariadicWriteFunc</i>(<i>SinkType</i> sink, const Args & ...args);</pre>

Table 2.2: Conceptual read/write functions in all primary I/O classes

Table 2.3 maps the generic conceptual interface to each of the primary I/O classes.

	Dwm I/O class				
	StreamIO	FileIO	DescriptorIO	BZ2IO	GZIO
<i>ReadFunc</i>	Read	Read	Read	BZRead	Read
<i>VariadicReadFunc</i>	ReadV	ReadV	ReadV	BZReadV	ReadV
<i>ReadReturnType</i>	std::istream &	size_t	ssize_t	int	int
<i>SourceType</i>	std::istream &	FILE *	int	BZFILE *	gzFile
<i>WriteFunc</i>	Write	Write	Write	BZWrite	Write
<i>VariadicWriteFunc</i>	WriteV	WriteV	WriteV	BZWriteV	WriteV
<i>WriteReturnType</i>	std::ostream &	size_t	ssize_t	int	int
<i>SinkType</i>	std::ostream &	FILE *	int	BZFILE *	gzFile

Table 2.3: Conceptual interface mapping to the primary I/O classes

2.3 Categories of interface overloads in actual implementation

There are three categories of overloads in the implementation:

- Regular functions for directly supported types
- Function templates for STL containers and other standard class templates
- Function templates for types that meet the conceptual requirements of a given [primary I/O class](#)

2.3.1 Regular functions for directly supported types

Reading and writing of directly supported types is done with regular function overloads. For example, this function from [Dwm::StreamIO](#) to write a `uint32_t` value:

```
static std::ostream & Write(std::ostream & os, uint32_t value);
```

2.3.2 Function templates for STL containers and other standard class templates

Overloaded function templates are used to read and write instantiations of standard class templates, where the parameter types of the standard class templates are either directly supported types, types that meet the conceptual requirements of a readable and writable class, or instances of supported standard templates using any of the aforementioned types in type parameters. In other words, supported standard containers of any depth can be read and written, as long as the leaf types are directly supported types or types that meet the conceptual requirements of a readable and writable class for the I/O class of interest.

This is probably best explained via some example readable and writable types:

```
std::vector<std::string>
std::map<uint32_t, std::vector<std::string>>
std::map<uint32_t, std::vector<std::map<uint8_t, std::string>>>
```

An example of one such function template from [Dwm::StreamIO](#):

```
template <typename valueT, size_t N>
static std::istream & Read(std::istream & is, std::array<valueT, N> & a)
```

2.3.3 Function templates for types that meet the conceptual requirements of a given primary I/O class

Overloaded function templates with concept requirements are used to read and write types that meet the concept requirements. The concepts required for each primary I/O class are listed in table 2.4.

An example of one such function template from `Dwm::StreamIO`:

```
static std::ostream & Write(std::ostream & os, const HasStreamWrite auto & t)
```

Dwm I/O class	Read concept	Write concept
<code>StreamIO</code>	<code>Dwm::HasStreamRead<T></code>	<code>Dwm::HasStreamWrite<T></code>
<code>FileIO</code>	<code>Dwm::HasFileRead<T></code>	<code>Dwm::HasFileWrite</t></code>
<code>DescriptorIO</code>	<code>Dwm::HasDescriptorRead<T></code>	<code>Dwm::HasDescriptorWrite<T></code>
<code>BZ2IO</code>	<code>Dwm::HasBZRead<T></code>	<code>Dwm::HasBZWrite<T></code>
<code>GZIO</code>	<code>Dwm::HasGZRead<T></code>	<code>Dwm::HasGZWrite<T></code>

Table 2.4: Primary I/O class concepts for user-defined types

2.4 Dwm::StreamIO read/write functions

2.4.1 Dwm::StreamIO conceptual read/write functions

Table 2.5 lists the conceptual read and write member functions in `Dwm::StreamIO`.

Primary I/O class	Read/write functions (conceptual)
<code>Dwm::StreamIO</code>	template <typename T> static std::istream & Read(std::istream & is, T & t);
	template <typename T> static std::ostream & Write(std::ostream & os, const T & t);
	template <typename... Args> static std::istream & ReadV(std::istream & is, Args & ...args);
	template <typename... Args> static std::ostream & WriteV(std::ostream & os, const Args & ...args);

Table 2.5: Conceptual read and write member functions in `Dwm::StreamIO`

Conceptually...

```
template <typename T>
static std::istream & Read(std::istream & is, T & t)
```

- Reads the contents of `t` from `is` and returns `is`.

```
template <typename T>
static std::ostream & Write(std::ostream & os, const T & t)
```

- Writes the contents of `t` to `os` and returns `os`.

```
template <typename... Args>
static std::istream & ReadV(std::istream & is, Args & ...args)
```

- Reads the contents of all `args` (in order) from `is` and returns `is`.

```
template <typename... Args>
static std::ostream & WriteV(std::ostream & os, const Args & ...args)
```

- Writes the contents of all `args` (in order) to `os` and returns `os`.

2.5 Dwm::FileIO read/write functions

2.5.1 Dwm::FileIO conceptual read/write functions

Primary I/O class	Read/write functions (conceptual)
Dwm::FileIO	template <typename T>
	static size_t Read(FILE *f, T & t);
	template <typename T>
	static size_t Write(FILE *f, const T & t);
	template <typename... Args>
	static size_t ReadV(FILE *f, Args & ...args);
	template <typename... Args>
	static size_t WriteV(FILE *f, const Args & ...args);

2.6 Dwm::DescriptorIO read/write functions

2.6.1 Dwm::DescriptorIO conceptual read/write functions

Primary I/O class	Read/write functions (conceptual)
Dwm::DescriptorIO	template <typename T>
	static ssize_t Read(int fd, T & t);
	template <typename T>
	static ssize_t Write(int fd, const T & t);
	template <typename... Args>
	static ssize_t ReadV(int fd, Args & ...args);
	template <typename... Args>
	static ssize_t WriteV(int fd, const Args & ...args);

2.7 Dwm::BZ2IO read/write functions

2.7.1 Dwm::BZ2IO conceptual read/write functions

Primary I/O class	Read/write functions (conceptual)
Dwm::BZ2IO	template <typename T>
	static int BZRead(BZFILE *bzf, T & t);
	template <typename T>
	static int BZWrite(BZFILE *bzf, const T & t)
	template <typename... Args>
	static int BZReadV(BZFILE *bzf, Args & ...args);
	template <typename T>
	static int BZWriteV(BZFILE *bzf, const Args & ...args)

2.8 Dwm::GZIO read/write functions

2.8.1 Dwm::GZIO conceptual read/write functions

Primary I/O class	Read/write functions (conceptual)
Dwm::GZIO	template <typename T>
	static int Read(gzFile gzf, T & t);
	template <typename T>
	static int Write(gzFile gzf, const T & t);
	template <typename... Args>
	static int ReadV(gzFile gzf, Args & ...args);
	template <typename... Args>
	static int Write(gzFile gzf, const Args & ...args);

2.9 Member function overloads

libDwm uses member function overloading to allow the user to use the same function names to read or write different types from/to a source/sink. This is also critical to supporting instances of class templates.

2.10 Member function templates

libDwm uses function templates and function template overloads to provide the same interfaces (the same member function names) to read or write instances of standard class templates from/to a source/sink (std::vector, std::pair, std::tuple, std::map, et. al.).

2.11 Concepts

libDwm uses concepts in many places to aid in overload resolution and to document the requirements of user-defined types.

2.12 Support for user-designed types

2.12.1 Before C++26

In general, code utilized with pre-C++26 compilers and standard libraries requires a pair of member functions to be added to any user-defined type for a given source/sink pair. Much of the time, these functions are fairly simple to implement, particularly for C++ iostreams (std::istream as a source, std::ostream as a sink). This is especially true for user-defined types that only contain types already supported by the corresponding primary I/O class (for C++ iostreams, that would be Dwm::StreamIO).

2.12.2 C++26 and beyond

With C++26 features (particularly P2996 and P3394), it becomes possible to elegantly handle serialization and deserialization of many user defined types with no changes to user code. It also allows annotations of types and variables that can control the serialization and deserialization of a user defined type (for example, skipping a data member of a class during serialization and deserialization). Work here has already begun, using a slightly modified fork of the bloomberg clang implementation.

Chapter 3

Getting Started

3.1 Write and read a string to/from an iostream

The program below writes a `std::string` to a `std::stringstream`, then reads a `std::string` from the `std::stringstream`. The program in listing [3.1](#) returns 0 if the string we read matches the string we wrote.

```
1 #include <sstream>
2 #include "DwmStreamIO.hh"
3
4 int main(int argc, char *argv[])
5 {
6     using std::string, std::stringstream;
7     bool      success{false};
8     string     writeString("Hello, world!"), readString;
9     stringstream ss;
10    if (Dwm::StreamIO::Write(ss, writeString)) {
11        if (Dwm::StreamIO::Read(ss, readString)) {
12            success = (readString == writeString);
13        }
14    }
15    return success ? 0 : 1;
16 }
```

Listing 3.1: Write and read a string to/from an iostream

3.2 Write and read a map<string,string> to/from an iostream

The program in listing 3.2 writes a `std::map<std::string,std::string>` to a `std::stringstream`, then reads a `std::map<std::string,std::string>` from the `std::stringstream` and returns 0 if the map we read matches the map we wrote. This is an example of our support for standard containers holding supported types (`std::string` is an explicitly supported type).

```
1 #include <sstream>
2 #include "DwmStreamIO.hh"
3
4 int main(int argc, char *argv[])
5 {
6     using stringmap = std::map<std::string,std::string>;
7     stringmap m1 {
8         { "apple",    "fruit"    },
9         { "cat",      "feline"   },
10        { "broccoli", "vegetable" },
11        { "dog",       "canine"   },
12        { "kiwi",      "fruit"    },
13        { "cow",       "bovine"   }
14    };
15    bool success{false};
16    std::stringstream ss;
17    if (Dwm::StreamIO::Write(ss, m1)) {
18        stringmap m2;
19        if (Dwm::StreamIO::Read(ss, m2)) {
20            success = (m2 == m1);
21        }
22    }
23    return success ? 0 : 1;
24 }
```

Listing 3.2: Write and read a map<string,string> to/from an iostream

3.3 Turtles all the way down

The program in listing 3.3 writes a `std::map<std::string, std::vector<std::string>>` to a `std::stringstream`, then reads a `std::map<std::string, std::vector<std::string>>` from the `std::stringstream` and returns 0 if the map we read matches the map we wrote. This is an example of our support for standard containers and supported types being “turtles all the way down”: our support is recursive to any depth.

```
1 #include <sstream>
2 #include "DwmStreamIO.hh"
3
4 int main(int argc, char *argv[])
5 {
6     using container = std::map<std::string, std::vector<std::string>>;
7     container m1 {
8         { "fruit",      { "apple", "banana", "kiwi", "orange" } },
9         { "canine",     { "husky", "labrador", "poodle", "wolf", "collie" } },
10        { "feline",      { "cougar", "lion", "panther" } },
11        { "vegetable", { "broccoli", "carrot", "squash" } }
12    };
13
14    bool success{false};
15    std::stringstream ss;
16    if (Dwm::StreamIO::Write(ss, m1)) {
17        container m2;
18        if (Dwm::StreamIO::Read(ss, m2)) {
19            success = (m2 == m1);
20        }
21    }
22    return success ? 0 : 1;
23 }
```

Listing 3.3: Write and read a `map<string, vector<string>>` to/from an `iostream`

3.4 Integrating a user-defined type

Listing 3.4 shows a user defined type `MyType` with three simple data members: `id`, `name` and `category`.

```

1 #include <cstdint>
2 #include <string>
3
4 struct MyType {
5     uint32_t    id;
6     std::string name;
7     std::string category;
8 };

```

Listing 3.4: `MyType` without explicit `Dwm::StreamIO` support

3.4.1 With C++26 reflection

With C++26 reflection, all of the [primary I/O classes](#) can read and write instances of `MyType` from/to their associated source/sink with no changes to `MyType`. The primary I/O classes will do what one would expect: read and write each of the members of `MyType`, in the order in which they appear in the declaration of `MyType`.

3.4.2 Without C++26 reflection

Without C++26 reflection, two members need to be added to `MyType` for each [primary I/O class](#) you wish to use with `MyType`. For example, listing 3.5 adds support for `Dwm::StreamIO` by adding the `Read(std::istream &)` and `Write(std::ostream &)` const member functions for the case where C++26 reflection is not available (`DWM_CAN_USE_REFLECTION` is not defined). Note that these member functions are simple in this case because our member data types are directly supported by `Dwm::StreamIO` and hence we can use the `ReadV()` and `WriteV()` members of `Dwm::StreamIO` to implement these members.

```

1 #include "DwmStreamIO.hh"
2
3 struct MyType {
4     uint32_t    id;
5     std::string name;
6     std::string category;
7
8     #if ! defined(DWM_CAN_USE_REFLECTION)
9         std::istream & Read(std::istream & is)
10        { return Dwm::StreamIO::ReadV(is, id, name, category); }
11
12        std::ostream & Write(std::ostream & os) const
13        { return Dwm::StreamIO::WriteV(os, id, name, category); }
14    #endif
15 };

```

Listing 3.5: `MyType` with explicit `Dwm::StreamIO` support when needed

3.4.3 A test program to write and read `std::vector<MyType>`

The program in listing 3.6 utilizes a user-defined type `MyType` that has explicit `Dwm::StreamIO` functionality as needed. `MyType` meets the requirements of the `HasStreamRead<T>` and `HasStreamWrite<T>` concepts by providing the `Read(std::istream &)` member at lines 10-11 and the `Write(std::ostream &) const` member at lines 13-14 when we do not have C++26 reflection.

```

1  #include <sstream>
2  #include "DwmStreamIO.hh"
3
4  struct MyType {
5      uint32_t    id;
6      std::string name;
7      std::string category;
8
9  #if ! defined(DWM_CAN_USE_REFLECTION)
10     std::istream & Read(std::istream & is)
11     { return Dwm::StreamIO::ReadV(is, id, name, category); }
12
13     std::ostream & Write(std::ostream & os) const
14     { return Dwm::StreamIO::WriteV(os, id, name, category); }
15 #endif
16
17     bool operator <=> (const MyType &) const = default; // default comparison
18 };
19
20 int main(int argc, char *argv[])
21 {
22     bool                rc{false};
23     std::vector<MyType> myvec1 {
24         {1, "cat", "feline"},
25         {2, "dog", "canine" },
26         {3, "cow", "bovine" }
27     };
28     std::stringstream    ss;
29     if (Dwm::StreamIO::Write(ss, myvec1)) {
30         std::vector<MyType> myvec2;
31         if (Dwm::StreamIO::Read(ss, myvec2)) {
32             rc = (myvec2 == myvec1);
33         }
34     }
35     return rc ? 0 : 1;
36 }

```

Listing 3.6: Integrating a user-defined type

The main function creates a vector of `MyType` named `myvec1` on line 23. It writes `myvec1` to the `stringstream` named `ss` at line 29. It then creates a second vector of `MyType` named `myvec2` on line 30 and reads the contents of `myvec2` from `ss` on line 31. On line 32 it checks if the vector we read (`myvec2`) matches the vector we wrote (`myvec1`).

3.4.4 Skipping data when using C++26 reflection

Chapter 4

Tips

4.1 Implement `iostream` support first

Make your type meet the requirements of `HasStreamRead<T>` and `HasStreamWrite<T>` first, by adding these two public members to your class or struct:

```
std::ostream & Write(std::ostream & os) const;  
std::istream & Read(std::istream & is);
```

The advantages to doing this before adding support for other source and sink types:

- Easier to unit test packing (write) and unpacking (read) correctness and symmetry, since `std::stringstream` may be used as source and sink (no interaction with the filesystem or network stack).
- Can later be used to shortcut support for sources and sinks other than `std::istream` and `std::ostream`.
- Can be leveraged to reduce read/write system calls (and potential context switches).
- Allows some resource tradeoffs when used to shortcut support for other sinks and sources.

4.2 Shortcuts to supporting other sinks/sources

4.3 Resource considerations

CPU consumption in user space

memory consumption

system calls like `read()` and `write()`

Chapter 5

Primary I/O Classes

There are several primary I/O classes in libDwm. Each is used with a particular type of source and sink.

`Dwm::StreamIO` is used with C++ iostreams (`std::istream` and `std::ostream`).

`Dwm::FileIO` is used with `FILE`s created via `fopen()`, `fdopen()`, et. al.

`Dwm::DescriptorIO` is used with UNIX file descriptors created via `open()`, et. al.

`Dwm::BZ2IO` is used with bz2 streams (BZFILES opened via `BZ2_bzopen()`).

`Dwm::GZIO` is used with gz streams (gzFiles opened via `gzopen()`).

When adding I/O support for your own data type (struct or class), `Dwm::StreamIO` support would typically be the first target. That's largely because for fixed length types, all of the other I/O classes can be supported via an intermediate pass through `Dwm::StreamIO` with a `std::stringstream` as a data buffer.

5.1 Dwm::StreamIO

source type: `std::istream`

sink type: `std::ostream`

`Dwm::StreamIO` provides static functions and function templates to read and write from/to C++ iostreams. Conceptually, the primary interface looks like four static member function templates:

```
namespace Dwm {
    class StreamIO {
    public:
        template <typename T>
        static std::istream & Read(std::istream & is, T & t);

        template <typename T>
        static std::ostream & Write(std::ostream & os, const T & t);

        template <typename... Args>
        static std::istream & ReadV(std::istream & is, Args & ...args);

        template <typename... Args>
        static std::ostream & WriteV(std::ostream & os, const Args & ...args);
    };
}
```

The actual implementation uses many function and function template overloads for `Read()` and `Write()`. It can't automatically read and write any type `T`; a data type must either be directly supported or meet the requirements of `Dwm::HasStreamRead` and `Dwm::HasStreamWrite`.

Note that `ReadV()` and `WriteV()` are just convenient variadic function templates that will read or write multiple objects in sequence.

5.1.1 Using `Dwm::StreamIO` with user defined types

If a user-defined type provides the following two public members, `Dwm::StreamIO` will utilize them to read and write instances of the user defined type:

```
std::istream & Read(std::istream & is)
std::ostream & Write(std::ostream & os) const
```

See the `Dwm::HasStreamRead` and `Dwm::HasStreamWrite` concepts for the semantic requirements of these members.

5.2 Dwm::FileIO

source type: FILE *

sink type: FILE *

Dwm::FileIO provides static functions and function templates to read and write from/to FILES. Conceptually, the interface looks like four static member function templates:

```
namespace Dwm {
    class FileIO {
    public:
        template <typename T>
        static size_t Read(FILE *f, T & t);

        template <typename T>
        static size_t Write(FILE *f, const T & t);

        template <typename... Args>
        static size_t ReadV(FILE *f, Args & ...args);

        template <typename... Args>
        static size_t WriteV(FILE *f, const Args & ...args);
    };
}
```

The actual implementation uses many function and function template overloads for Read() and Write(). It can't automatically read and write any type T; a data type must either be directly supported or meet the requirements of [Dwm::HasFileRead](#) and [Dwm::HasFileWrite](#).

Note that ReadV() and WriteV() are just convenient variadic function templates that will read or write multiple objects in sequence.

5.2.1 Using Dwm::FileIO with user defined types

If a user-defined type provides the following two public members, Dwm::FileIO will utilize them to read and write instances of the user defined type:

```
size_t Read(FILE *f)
size_t Write(FILE *f) const
```

See the [Dwm::HasFileRead](#) and [Dwm::HasFileWrite](#) concepts for the semantic requirements of these members.

5.3 Dwm::DescriptorIO

source type: int
sink type: int

Dwm::DescriptorIO provides static functions and function templates to read and write from/to UNIX file descriptors. Conceptually, the interface looks like four static member function templates:

```
namespace Dwm {
    class DescriptorIO {
    public:
        <template typename T> static ssize_t Read(int fd, T & t);

        <template typename T> static ssize_t Write(int fd, const T & t);

        template <typename ...Args>
        static ssize_t ReadV(int fd, Args & ...args);

        template <typename ...Args>
        static ssize_t WriteV(int fd, const Args & ...args);
    };
}
```

The actual implementation uses many function and function template overloads for Read() and Write(). It can't automatically read and write any type T; a data type must either be directly supported or meet the requirements of Dwm::HasDescriptorRead and Dwm::HasDescriptorWrite.

Note that ReadV() and WriteV() are just convenient variadic function templates that will read or write multiple objects in sequence.

5.3.1 Using Dwm::DescriptorIO with user defined types

If a user defined type provides the following two public members, Dwm::DescriptorIO will utilize them to read and write instances of the user defined type:

```
ssize_t Read(int fd)
ssize_t Write(int fd) const
```

See the Dwm::HasDescriptorRead and Dwm::HasDescriptorWrite concepts for the semantic requirements of these members.

5.4 Dwm::BZ2IO

source type: BZFILE *

sink type: BZFILE *

Dwm::BZ2IO provides static functions and function templates to read and write from/to bz2 streams (BZFILE *). Conceptually, the interface looks like four static member function templates:

```
namespace Dwm {
    class BZ2IO {
    public:
        <template typename T>
        static int BZRead(BZFILE *bzf, T & t);

        <template typename T>
        static int BZWrite(BZFILE *bzf, const T & t);

        template <typename ...Args>
        static int BZReadV(BZFILE *bzf, Args & ...args);

        template <typename ...Args>
        static int BZWriteV(BZFILE *bzf, const Args & ...args);
    };
}
```

The actual implementation uses many function and function template overloads for BZRead() and BZWrite(). It can't automatically read and write any type T; a data type must either be directly supported or meet the requirements of [Dwm::HasBZRead](#) and [Dwm::HasBZWrite](#).

5.4.1 Using Dwm::BZ2IO with user defined types

If a user defined type provides the following two public members, Dwm::BZ2IO will utilize them to read and write instances of the user defined type:

```
int BZRead(BZFILE *)
int BZWrite(BZFILE *) const
```

See the [Dwm::HasBZRead](#) and [Dwm::HasBZWrite](#) concepts for the semantic requirements of these member functions.

5.5 Dwm::GZIO

gzFile source type: gzFile
 sink type: gzFile

Dwm::GZIO provides static functions and function templates to read and write from/to gz streams (gzFile). Conceptually, the interface looks like four static member function templates:

```
namespace Dwm {
    class GZIO {
    public:
        <template typename T>
        static int Read(gzFile gzf, T & t);

        <template typename T>
        static int Write(gzFile gzf, const T & t);

        template <typename ...Args>
        static int ReadV(gzFile gzf, Args & ...args);

        template <typename ...Args>
        static int WriteV(gzFile gzf, const Args & ...args);
    };
}
```

The actual implementation uses many function and function template overloads for Read() and Write(). It can't automatically read and write any type T; a data type must either be directly supported or meet the requirements of [Dwm::HasGZRead](#) and [Dwm::HasGZWrite](#).

5.5.1 Using Dwm::GZIO with user defined types

If a user defined type provides the following two public members, Dwm::GZIO will utilize them to read and write instances of the user defined type:

```
int Read(gzFile)
int Write(gzFile) const
```

See the [Dwm::HasGZRead](#) and [Dwm::HasGZWrite](#) concepts for the expected semantics of these members.

5.6 Dwm::ASIO

Boost asio sockets

Chapter 6

Other I/O classes, functions and templates

6.1 `Dwm::EncodedUnsigned<T>`

`Dwm::EncodedUnsigned<T>` is a class template for encoding and decoding unsigned integers of 16, 32 and 64 bits. The class template automatically handles endian conversions as needed when writing and reading values of these types.

Three class template instantiations are provided:

```
Dwm::EncodedU64 → Dwm::EncodedUnsigned<uint64_t>  
Dwm::EncodedU32 → Dwm::EncodedUnsigned<uint32_t>  
Dwm::EncodedU16 → Dwm::EncodedUnsigned<uint16_t>
```

6.1.1 `Dwm::EncodedU64`

`Dwm::EncodedU64` represents a `uint64_t` value, but on the wire is encoded as an encoding byte followed by a variable length (1 to 8) sequence of bytes containing the value. It is a typical space/time tradeoff: it requires more time to read or write, but saves space on the wire and storage for some common use cases.

Common use cases generally revolve around `std::size_t` values representing the length of a `std::string`, the number of elements in an STL container (vector, map, et. al.), or the return of the `sizeof` operator. On a 64-bit platform, you may need to accomodate 64-bit length values, but find that all of your values are less than 2^{32} . Or less than 256. These are scenarios where `Dwm::EncodedU64` may be useful if the space/time tradeoff is desired.

See [Dwm::EncodedU64 encoding](#) for details on the encoding format.

6.1.2 `Dwm::EncodedU32`

`Dwm::EncodedU32` represents a `uint32_t` value, but on the wire is encoded as an encoding byte followed by a variable length (1 to 4) sequence of bytes containing the value. It is a typical space/time tradeoff: it requires more time to read or write, but may save space on the wire and storage for some common use cases and also automatically handles endian conversion when reading and writing.

See [Dwm::EncodedU32 encoding](#) for details on the encoding format.

6.1.3 `Dwm::EncodedU16`

`Dwm::EncodedU16` represents a `uint16_t` value, but on the wire is encoded as an encoding byte followed by a variable length (1 to 2) sequence of bytes containing the value.

Unlike [Dwm::EncodedU32](#) and [Dwm::EncodedU64](#), no bandwidth is saved versus using `uint16_t`: at least 2

bytes are always written or read (1 byte for encoding and at least 1 byte for the value). The only reason to use this class versus `uint16_t` when reading and writing `uint16_t` values is the automatic endian conversion handling.

See [Dwm::EncodedU16 encoding](#) for details on the encoding format.

6.2 Dwm::EncodedSigned<T>

`Dwm::EncodedSigned<T>` is a class template for encoding and decoding signed integers of 16, 32 and 64 bits. The class template automatically handles endian conversions as needed when writing and reading values of these types.

Three class template instantiations are provided:

```
Dwm::EncodedS64 → Dwm::EncodedSigned<int64_t>
Dwm::EncodedS32 → Dwm::EncodedSigned<int32_t>
Dwm::EncodedS16 → Dwm::EncodedSigned<int16_t>
```

6.2.1 Dwm::EncodedS64

`Dwm::EncodedS64` represents an `int64_t` value, but on the wire is encoded as an encoding byte followed by a variable length (1 to 8) sequence of bytes containing the value. It is a typical space/time tradeoff: it requires more time to read or write, but saves space on the wire and storage for some common use cases. It also automatically handles endian conversion when reading and writing.

See [Dwm::EncodedS64 encoding](#) for details on the encoding format.

6.2.2 Dwm::EncodedS32

`Dwm::EncodedS32` represents an `int32_t` value, but on the wire is encoded as an encoding byte followed by a variable length (1 to 4) sequence of bytes containing the value. It is a typical space/time tradeoff: it requires more time to read or write, but may save space on the wire and storage for some common use cases and also automatically handles endian conversion when reading and writing.

See [Dwm::EncodedS32 encoding](#) for details on the encoding format.

6.2.3 Dwm::EncodedS16

`Dwm::EncodedS16` represents an `int16_t` value, but on the wire is encoded as an encoding byte followed by a variable length (1 to 2) sequence of bytes containing the value. It automatically handles endian conversion when reading and writing. Writes are always done in native endian format. Reads will automatically convert to native endian format.

Unlike [Dwm::EncodedS32](#) and [Dwm::EncodedS64](#), no bandwidth is saved versus using `int16_t`: at least 2 bytes are always written or read (1 byte for encoding and at least 1 byte for the value). The only reason to use this class versus `int16_t` when reading and writing `int16_t` values is the automatic endian conversion handling.

See [Dwm::EncodedS16 encoding](#) for details on the encoding format.

6.3 Dwm::IOUtils

6.4 Endian conversions

6.4.1 `template <typename T>` `concept IsEndianSensitiveInteger`

True if `std::is_integral_v<T>` and `sizeof(T)` is 2, 4 or 8.

Examples:

```
IsEndianSensitiveInteger<uint64_t> → true
IsEndianSensitiveInteger<uint32_t> → true
IsEndianSensitiveInteger<uint16_t> → true
IsEndianSensitiveInteger<uint8_t> → false
IsEndianSensitiveInteger<char> → false
IsEndianSensitiveInteger<float> → false
```

6.4.2 BE2Host

```
template <typename T>
requires IsEndianSensitiveInteger<T>
inline auto BE2Host(T t) → T
```

Returns a T holding the value of t converted from big endian (MSB first) byte order to host byte order. Example:

```
uint16_t x = 0xFF00;
uint16_t y = Dwm::BE2Host(x);
// on a little endian machine, y == 0x00FF
assert(0x00FF == y);
```

```
template <typename ...Args>
requires (IsEndianSensitiveInteger<Args> and ...)
void BE2Host(Args & ...args)
```

TODO: Document BE2Host variadic

6.4.3 Host2BE

```
template <typename T>
requires IsEndianSensitiveInteger<T>
inline auto Host2BE(T t) → T
```

Returns a T holding the value of t converted from host byte order to big-endian (MSB first) byte order. Example:

```
uint32_t x = 0xAABBCCDD;
uint32_t y = Dwm::Host2BE(x);
// on a little endian machine, y == 0xDDCCBBAA
assert(0xDDCCBBAA == y);
```

```
template <typename ...Args>
```

```
requires (IsEndianSensitiveInteger<Args> and ...)  
void Host2BE(Args & ...args)
```

TODO: Document Host2BE variadic

6.4.4 Bswap

TODO: Document Bswap

Chapter 7

Directly supported data types

Directly supported data types are those types for which we have static member functions to read the type from a source and write the type to a sink. These types include integral types, floating point types, enumerated types, `std::string` and `std::vector<bool>`.

7.1 integral types

The category of types for which `std::is_integral_v<T>` evaluates to true.

Size	Supported type	typically aliased from
1 byte	<code>bool</code>	
	<code>char</code>	
	<code>int8_t</code>	<code>char</code>
	<code>uint8_t</code>	<code>unsigned char</code>
2 bytes	<code>int16_t</code>	<code>short</code>
	<code>uint16_t</code>	<code>unsigned short</code>
4 bytes	<code>int32_t</code>	<code>int</code> or <code>long</code>
	<code>uint32_t</code>	<code>unsigned int</code> or <code>unsigned long</code>
8 bytes	<code>int64_t</code>	<code>long long</code>
	<code>uint64_t</code>	<code>unsigned long long</code>

For multi-byte types in this category, we use an MSB-first representation on-the-wire.

7.2 floating point types

The category of types for which `std::is_floating_point_v<T>` evaluates to true.

Size	Supported type
4 bytes	<code>float</code>
8 bytes	<code>double</code>

We use XDR for the on-the-wire representation of `float` and `double`.

7.3 enumerated types

The category of types for which `std::is_enum_v<T>` evaluates to true. Note that in general, we expect an enumerated type to have an explicit underlying type so we are certain of its size.

7.4 `std::string`

`std::string` is explicitly supported.

7.5 `std::vector<bool>`

`std::vector<bool>` is explicitly supported, mainly because it's not a conforming container (which all of us in the C++ community wish was not true). In general it's unwise to use `std::vector<bool>` due to its lack of complete conformance with what's expected of an instance of `std::vector<T>`.

Chapter 8

Supported `std` type templates

8.1 `std::array<class T, std::size_t N>`

We can stream any instances of `std::array<T,N>` where `T` satisfies the read and write concepts of the desired primary I/O class.

primary I/O class	Read concept	Write concept
<code>Dwm::StreamIO</code>	<code>Dwm::IsStreamReadable<T></code>	<code>Dwm::IsStreamWritable<T></code>
<code>Dwm::FileIO</code>	<code>Dwm::IsFileReadable<T></code>	<code>Dwm::IsFileWritable<T></code>
<code>Dwm::DescriptorIO</code>	<code>Dwm::IsDescriptorReadable<T></code>	<code>Dwm::IsDescriptorWritable<T></code>
<code>Dwm::BZ2IO</code>	<code>Dwm::IsBZ2Readable<T></code>	<code>Dwm::IsBZ2Writable<T></code>
<code>Dwm::GZIO</code>	<code>Dwm::IsGZReadable<T></code>	<code>Dwm::IsGZWritable<T></code>

8.2 `std::atomic<class T>`

We can stream any instances of `std::atomic<T>` where `T` satisfies the read and write concepts of the desired primary I/O class. Note this excludes cases where `T` is a raw pointer, `std::shared_ptr` or `std::weak_ptr`.

primary I/O class	Read concept	Write concept
<code>Dwm::StreamIO</code>	<code>Dwm::IsStreamReadable<T></code>	<code>Dwm::IsStreamWritable<T></code>
<code>Dwm::FileIO</code>	<code>Dwm::IsFileReadable<T></code>	<code>Dwm::IsFileWritable<T></code>
<code>Dwm::DescriptorIO</code>	<code>Dwm::IsDescriptorReadable<T></code>	<code>Dwm::IsDescriptorWritable<T></code>
<code>Dwm::BZ2IO</code>	<code>Dwm::IsBZ2Readable<T></code>	<code>Dwm::IsBZ2Writable<T></code>
<code>Dwm::GZIO</code>	<code>Dwm::IsGZReadable<T></code>	<code>Dwm::IsGZWritable<T></code>

8.3 `std::deque<class T>`

We can stream any instances of `std::deque<T>` where `T` satisfies the read and write concepts of the desired primary I/O class.

primary I/O class	Read concept	Write concept
<code>Dwm::StreamIO</code>	<code>Dwm::IsStreamReadable<T></code>	<code>Dwm::IsStreamWritable<T></code>
<code>Dwm::FileIO</code>	<code>Dwm::IsFileReadable<T></code>	<code>Dwm::IsFileWritable<T></code>
<code>Dwm::DescriptorIO</code>	<code>Dwm::IsDescriptorReadable<T></code>	<code>Dwm::IsDescriptorWritable<T></code>
<code>Dwm::BZ2IO</code>	<code>Dwm::IsBZ2Readable<T></code>	<code>Dwm::IsBZ2Writable<T></code>
<code>Dwm::GZIO</code>	<code>Dwm::IsGZReadable<T></code>	<code>Dwm::IsGZWritable<T></code>

See `std::deque<class T> encoding` for encoding detail.

8.4 `std::list<class T>`

We can stream any instances of `std::list<T>` where `T` satisfies the read and write concepts of the desired primary I/O class.

primary I/O class	Read concept	Write concept
<code>Dwm::StreamIO</code>	<code>Dwm::IsStreamReadable<T></code>	<code>Dwm::IsStreamWritable<T></code>
<code>Dwm::FileIO</code>	<code>Dwm::IsFileReadable<T></code>	<code>Dwm::IsFileWritable<T></code>
<code>Dwm::DescriptorIO</code>	<code>Dwm::IsDescriptorReadable<T></code>	<code>Dwm::IsDescriptorWritable<T></code>
<code>Dwm::BZ2IO</code>	<code>Dwm::IsBZ2Readable<T></code>	<code>Dwm::IsBZ2Writable<T></code>
<code>Dwm::GZIO</code>	<code>Dwm::IsGZReadable<T></code>	<code>Dwm::IsGZWritable<T></code>

See `std::list<class T>` [encoding](#) for encoding detail.

8.5 `std::map<KeyType, MappedType>`

We can stream any instances of `std::map<KeyType, MappedType>` where `KeyType` and `MappedType` satisfy the read and write requirements of the desired primary I/O class.

primary I/O class	Read concept	Write concept
<code>Dwm::StreamIO</code>	<code>Dwm::IsStreamReadable<KeyType></code>	<code>Dwm::IsStreamWritable<KeyType></code>
	<code>Dwm::IsStreamReadable<MappedType></code>	<code>Dwm::IsStreamWritable<MappedType></code>
<code>Dwm::FileIO</code>	<code>Dwm::IsFileReadable<KeyType></code>	<code>Dwm::IsFileWritable<KeyType></code>
	<code>Dwm::IsFileReadable<MappedType></code>	<code>Dwm::IsFileWritable<MappedType></code>
<code>Dwm::DescriptorIO</code>	<code>Dwm::IsDescriptorReadable<KeyType></code>	<code>Dwm::IsDescriptorWritable<KeyType></code>
	<code>Dwm::IsDescriptorReadable<MappedType></code>	<code>Dwm::IsDescriptorWritable<MappedType></code>
<code>Dwm::BZ2IO</code>	<code>Dwm::IsBZ2Readable<KeyType></code>	<code>Dwm::IsBZ2Writable<KeyType></code>
	<code>Dwm::IsBZ2Readable<MappedType></code>	<code>Dwm::IsBZ2Writable<MappedType></code>
<code>Dwm::GZIO</code>	<code>Dwm::IsGZReadable<KeyType></code>	<code>Dwm::IsGZWritable<KeyType></code>
	<code>Dwm::IsGZReadable<MappedType></code>	<code>Dwm::IsGZWritable<MappedType></code>

See `std::map<class KeyType, class MappedType>` [encoding](#) for encoding detail.

8.6 `std::multimap<KeyType, MappedType>`

We can stream any instances of `std::multimap<KeyType, MappedType>` where `KeyType` and `MappedType` satisfy the read and write requirements of the desired primary I/O class.

primary I/O class	Read concept	Write concept
<code>Dwm::StreamIO</code>	<code>Dwm::IsStreamReadable<KeyType></code>	<code>Dwm::IsStreamWritable<KeyType></code>
	<code>Dwm::IsStreamReadable<MappedType></code>	<code>Dwm::IsStreamWritable<MappedType></code>
<code>Dwm::FileIO</code>	<code>Dwm::IsFileReadable<KeyType></code>	<code>Dwm::IsFileWritable<KeyType></code>
	<code>Dwm::IsFileReadable<MappedType></code>	<code>Dwm::IsFileWritable<MappedType></code>
<code>Dwm::DescriptorIO</code>	<code>Dwm::IsDescriptorReadable<KeyType></code>	<code>Dwm::IsDescriptorWritable<KeyType></code>
	<code>Dwm::IsDescriptorReadable<MappedType></code>	<code>Dwm::IsDescriptorWritable<MappedType></code>
<code>Dwm::BZ2IO</code>	<code>Dwm::IsBZ2Readable<KeyType></code>	<code>Dwm::IsBZ2Writable<KeyType></code>
	<code>Dwm::IsBZ2Readable<MappedType></code>	<code>Dwm::IsBZ2Writable<MappedType></code>
<code>Dwm::GZIO</code>	<code>Dwm::IsGZReadable<KeyType></code>	<code>Dwm::IsGZWritable<KeyType></code>
	<code>Dwm::IsGZReadable<MappedType></code>	<code>Dwm::IsGZWritable<MappedType></code>

See `std::multimap<class KeyType, class MappedType>` [encoding](#) for encoding detail.

8.7 `std::optional<class T>`

We can stream any instances of `std::optional<T>` where `T` satisfies the read and write concepts of the desired primary I/O class.

primary I/O class	Read concept	Write concept
<code>Dwm::StreamIO</code>	<code>Dwm::IsStreamReadable<T></code>	<code>Dwm::IsStreamWritable<T></code>
<code>Dwm::FileIO</code>	<code>Dwm::IsFileReadable<T></code>	<code>Dwm::IsFileWritable<T></code>
<code>Dwm::DescriptorIO</code>	<code>Dwm::IsDescriptorReadable<T></code>	<code>Dwm::IsDescriptorWritable<T></code>
<code>Dwm::BZ2IO</code>	<code>Dwm::IsBZ2Readable<T></code>	<code>Dwm::IsBZ2Writable<T></code>
<code>Dwm::GZIO</code>	<code>Dwm::IsGZReadable<T></code>	<code>Dwm::IsGZWritable<T></code>

See `std::optional<T>` [encoding](#) for encoding details.

8.8 `std::pair<class FirstType, class SecondType>`

We can stream any instances of `std::pair<FirstType, SecondType>` where `FirstType` and `SecondType` satisfy the read and write concepts of the desired primary I/O class.

primary I/O class	Read concept	Write concept
<code>Dwm::StreamIO</code>	<code>Dwm::IsStreamReadable<FirstType></code> <code>Dwm::IsStreamReadable<SecondType></code>	<code>Dwm::IsStreamWritable<FirstType></code> <code>Dwm::IsStreamWritable<SecondType></code>
<code>Dwm::FileIO</code>	<code>Dwm::IsFileReadable<FirstType></code> <code>Dwm::IsFileReadable<SecondType></code>	<code>Dwm::IsFileWritable<FirstType></code> <code>Dwm::IsFileWritable<SecondType></code>
<code>Dwm::DescriptorIO</code>	<code>Dwm::IsDescriptorReadable<FirstType></code> <code>Dwm::IsDescriptorReadable<SecondType></code>	<code>Dwm::IsDescriptorWritable<FirstType></code> <code>Dwm::IsDescriptorWritable<SecondType></code>
<code>Dwm::BZ2IO</code>	<code>Dwm::IsBZ2Readable<FirstType></code> <code>Dwm::IsBZ2Readable<SecondType></code>	<code>Dwm::IsBZ2Writable<FirstType></code> <code>Dwm::IsBZ2Writable<SecondType></code>
<code>Dwm::GZIO</code>	<code>Dwm::IsGZReadable<FirstType></code> <code>Dwm::IsGZReadable<SecondType></code>	<code>Dwm::IsGZWritable<FirstType></code> <code>Dwm::IsGZWritable<SecondType></code>

See `std::pair<class FirstType, class FirstType>` [encoding](#) for encoding details.

8.9 `std::set<class T>`

We can stream any instances of `std::set<T>` where `T` satisfies the read and write concepts of the desired primary I/O class.

primary I/O class	Read concept	Write concept
<code>Dwm::StreamIO</code>	<code>Dwm::IsStreamReadable<T></code>	<code>Dwm::IsStreamWritable<T></code>
<code>Dwm::FileIO</code>	<code>Dwm::IsFileReadable<T></code>	<code>Dwm::IsFileWritable<T></code>
<code>Dwm::DescriptorIO</code>	<code>Dwm::IsDescriptorReadable<T></code>	<code>Dwm::IsDescriptorWritable<T></code>
<code>Dwm::BZ2IO</code>	<code>Dwm::IsBZ2Readable<T></code>	<code>Dwm::IsBZ2Writable<T></code>
<code>Dwm::GZIO</code>	<code>Dwm::IsGZReadable<T></code>	<code>Dwm::IsGZWritable<T></code>

See `std::set<class T>` [encoding](#) for encoding details.

8.10 `std::multiset<class T>`

We can stream any instances of `std::multiset<T>` where `T` satisfies the read and write concepts of the desired primary I/O class.

primary I/O class	Read concept	Write concept
<code>Dwm::StreamIO</code>	<code>Dwm::IsStreamReadable<T></code>	<code>Dwm::IsStreamWritable<T></code>
<code>Dwm::FileIO</code>	<code>Dwm::IsFileReadable<T></code>	<code>Dwm::IsFileWritable<T></code>
<code>Dwm::DescriptorIO</code>	<code>Dwm::IsDescriptorReadable<T></code>	<code>Dwm::IsDescriptorWritable<T></code>
<code>Dwm::BZ2IO</code>	<code>Dwm::IsBZ2Readable<T></code>	<code>Dwm::IsBZ2Writable<T></code>
<code>Dwm::GZIO</code>	<code>Dwm::IsGZReadable<T></code>	<code>Dwm::IsGZWritable<T></code>

See `std::multiset<class T>` [encoding](#) for encoding detail.

8.11 `std::unique_ptr<class T>`

We can stream instances of `std::unique_ptr<T>` where `T` is streamable and the `unique_ptr`'s `deleter_type` is `std::default_delete<T>`. We use the `deleter_type` test to infer that the `std::unique_ptr<T>` points to a single object and not to an array. This can of course be incorrect, and hence should not be relied on as a general mechanism.

8.12 `std::unordered_map<class KeyType, class MappedType>`

We can stream any instances of `std::unordered_map<KeyType, MappedType>` where `KeyType` and `MappedType` satisfy the read and write requirements of the desired primary I/O class.

primary I/O class	Read concept	Write concept
<code>Dwm::StreamIO</code>	<code>Dwm::IsStreamReadable<KeyType></code> <code>Dwm::IsStreamReadable<MappedType></code>	<code>Dwm::IsStreamWritable<KeyType></code> <code>Dwm::IsStreamWritable<MappedType></code>
<code>Dwm::FileIO</code>	<code>Dwm::IsFileReadable<KeyType></code> <code>Dwm::IsFileReadable<MappedType></code>	<code>Dwm::IsFileWritable<KeyType></code> <code>Dwm::IsFileWritable<MappedType></code>
<code>Dwm::DescriptorIO</code>	<code>Dwm::IsDescriptorReadable<KeyType></code> <code>Dwm::IsDescriptorReadable<MappedType></code>	<code>Dwm::IsDescriptorWritable<KeyType></code> <code>Dwm::IsDescriptorWritable<MappedType></code>
<code>Dwm::BZ2IO</code>	<code>Dwm::IsBZ2Readable<KeyType></code> <code>Dwm::IsBZ2Readable<MappedType></code>	<code>Dwm::IsBZ2Writable<KeyType></code> <code>Dwm::IsBZ2Writable<MappedType></code>
<code>Dwm::GZIO</code>	<code>Dwm::IsGZReadable<KeyType></code> <code>Dwm::IsGZReadable<MappedType></code>	<code>Dwm::IsGZWritable<KeyType></code> <code>Dwm::IsGZWritable<MappedType></code>

See `std::unordered_map<class KeyType, class MappedType>` encoding for encoding detail.

8.13 `std::unordered_multimap<class KeyType, class MappedType>`

We can stream any instances of `std::unordered_multimap<KeyType, MappedType>` where `KeyType` and `MappedType` satisfy the read and write requirements of the desired primary I/O class.

primary I/O class	Read concept	Write concept
<code>Dwm::StreamIO</code>	<code>Dwm::IsStreamReadable<KeyType></code> <code>Dwm::IsStreamReadable<MappedType></code>	<code>Dwm::IsStreamWritable<KeyType></code> <code>Dwm::IsStreamWritable<MappedType></code>
<code>Dwm::FileIO</code>	<code>Dwm::IsFileReadable<KeyType></code> <code>Dwm::IsFileReadable<MappedType></code>	<code>Dwm::IsFileWritable<KeyType></code> <code>Dwm::IsFileWritable<MappedType></code>
<code>Dwm::DescriptorIO</code>	<code>Dwm::IsDescriptorReadable<KeyType></code> <code>Dwm::IsDescriptorReadable<MappedType></code>	<code>Dwm::IsDescriptorWritable<KeyType></code> <code>Dwm::IsDescriptorWritable<MappedType></code>
<code>Dwm::BZ2IO</code>	<code>Dwm::IsBZ2Readable<KeyType></code> <code>Dwm::IsBZ2Readable<MappedType></code>	<code>Dwm::IsBZ2Writable<KeyType></code> <code>Dwm::IsBZ2Writable<MappedType></code>
<code>Dwm::GZIO</code>	<code>Dwm::IsGZReadable<KeyType></code> <code>Dwm::IsGZReadable<MappedType></code>	<code>Dwm::IsGZWritable<KeyType></code> <code>Dwm::IsGZWritable<MappedType></code>

See `std::unordered_multimap<class KeyType, class MappedType>` encoding for encoding detail.

8.14 `std::unordered_set<class T>`

We can stream any instances of `std::unordered_set<T>` where `T` satisfies the read and write concepts of the desired primary I/O class.

primary I/O class	Read concept	Write concept
<code>Dwm::StreamIO</code>	<code>Dwm::IsStreamReadable<T></code>	<code>Dwm::IsStreamWritable<T></code>
<code>Dwm::FileIO</code>	<code>Dwm::IsFileReadable<T></code>	<code>Dwm::IsFileWritable<T></code>
<code>Dwm::DescriptorIO</code>	<code>Dwm::IsDescriptorReadable<T></code>	<code>Dwm::IsDescriptorWritable<T></code>
<code>Dwm::BZ2IO</code>	<code>Dwm::IsBZ2Readable<T></code>	<code>Dwm::IsBZ2Writable<T></code>
<code>Dwm::GZIO</code>	<code>Dwm::IsGZReadable<T></code>	<code>Dwm::IsGZWritable<T></code>

See `std::unordered_set<class T>` encoding for encoding detail.

8.15 `std::unordered_multiset<class T>`

We can stream any instances of `std::unordered_multiset<T>` where `T` satisfies the read and write concepts of the desired primary I/O class.

primary I/O class	Read concept	Write concept
<code>Dwm::StreamIO</code>	<code>Dwm::IsStreamReadable<T></code>	<code>Dwm::IsStreamWritable<T></code>
<code>Dwm::FileIO</code>	<code>Dwm::IsFileReadable<T></code>	<code>Dwm::IsFileWritable<T></code>
<code>Dwm::DescriptorIO</code>	<code>Dwm::IsDescriptorReadable<T></code>	<code>Dwm::IsDescriptorWritable<T></code>
<code>Dwm::BZ2IO</code>	<code>Dwm::IsBZ2Readable<T></code>	<code>Dwm::IsBZ2Writable<T></code>
<code>Dwm::GZIOW</code>	<code>Dwm::IsGZReadable<T></code>	<code>Dwm::IsGZWritable<T></code>

See `std::unordered_multiset<class T>` [encoding](#) for encoding detail.

8.16 `std::vector<class T>`

We can stream any instances of `std::vector<T>` where `T` satisfies the read and write concepts of the desired primary I/O class.

primary I/O class	Read concept	Write concept
<code>Dwm::StreamIO</code>	<code>Dwm::IsStreamReadable<T></code>	<code>Dwm::IsStreamWritable<T></code>
<code>Dwm::FileIO</code>	<code>Dwm::IsFileReadable<T></code>	<code>Dwm::IsFileWritable<T></code>
<code>Dwm::DescriptorIO</code>	<code>Dwm::IsDescriptorReadable<T></code>	<code>Dwm::IsDescriptorWritable<T></code>
<code>Dwm::BZ2IO</code>	<code>Dwm::IsBZ2Readable<T></code>	<code>Dwm::IsBZ2Writable<T></code>
<code>Dwm::GZIOW</code>	<code>Dwm::IsGZReadable<T></code>	<code>Dwm::IsGZWritable<T></code>

See `std::vector<class T>` [encoding](#) for encoding detail.

Chapter 9

Concepts

9.1 Streamable

9.2 Concepts used by `Dwm::StreamIO` (C++ iostreams)

9.2.1 `HasStreamRead<T>`

Evaluates to true if T has a member function with signature:

```
std::istream & Read(std::istream & is)
```

Required semantics

- Must populate `*this` with contents read from `is`
- Must return `is`

Other semantics

- May set `std::ios_base::failbit`

9.2.2 `HasStreamWrite<T>`

Evaluates to true if T has a member function with signature:

```
std::ostream & Write(std::ostream & os) const
```

Required semantics

- Must write the contents of `*this` to `os`
- Must return `os`

Other semantics

- May set `std::ios_base::failbit`

9.2.3 `Dwm::IsStreamReadable<T>`

Evaluates to true if T is a directly supported type, a type that satisfies `HasStreamRead<T>`, an instance of a supported class template, or (if C++26 reflection is available) is readable via reflection.

9.2.4 `Dwm::IsStreamWritable<T>`

Evaluates to true if T is a directly supported type, a type that satisfies `HasStreamWrite<T>`, an instance of a supported class template, or (if C++26 reflection is available) is writable via reflection.

9.3 Concepts used by `Dwm::FileIO` (FILES, i.e. `cstdio`)

9.3.1 `HasFileRead<T>`

Evaluates to true if T has a member function with signature:

```
size_t Read(FILE *f)
```

Required semantics

- Must populate `*this` with contents read from `f`
- Must return a non-zero `size_t` on success, 0 on failure

9.3.2 `HasFileWrite<T>`

Evaluates to true if T has a member function with signature:

```
size_t Write(FILE *f) const
```

Required semantics

- Must write the contents of `*this` to `f`
- Must return a non-zero `size_t` on success, 0 on failure

9.3.3 `Dwm::IsFileReadable<T>`

9.3.4 `Dwm::IsFileWritable<T>`

9.4 Concepts used by `Dwm::DescriptorIO` (UNIX descriptors)

9.4.1 `HasDescriptorRead<T>`

Evaluates to true if T has a member function with signature:

```
ssize_t Read(int fd)
```

Required semantics

- Must populate `*this` with contents read from `fd`
- Must return a positive `ssize_t` on success, -1 on failure

9.4.2 `HasDescriptorWrite<T>`

Evaluates to true if T has a member function with signature:

```
ssize_t Write(int fd) const
```

Required semantics

- Must write the contents of `*this` to `fd`
- Must return a positive `ssize_t` on success, -1 on failure

9.4.3 `Dwm::IsDescriptorReadable<T>`

9.4.4 `Dwm::IsDescriptorWritable<T>`

9.5 Boost asio sockets

9.5.1 HasAsioRead<T>

9.5.2 HasAsioWrite<T>

9.6 Concepts used by Dwm::BZ2IO (BZFILEs from BZ2_bzopen())

9.6.1 HasBZRead<T>

Evaluates to true if T has a member function with signature:

```
int BZRead(BZFILE *bzf)
```

Required semantics

- Must populate *this with contents read from bzf
- Must return a positive int on success, -1 on failure

9.6.2 HasBZWrite<T>

Evaluates to true if T has a member function with signature:

```
int Write(BZFILE *bzf) const
```

Required semantics

- Must write the contents of *this to bzf
- Must return a positive int on success, -1 on failure

9.6.3 Dwm::IsBZ2Readable<T>

9.6.4 Dwm::IsBZ2Writable<T>

9.7 Concepts used by Dwm::GZIO (gzFiles from gzopen())

9.7.1 HasGZRead<T>

Evaluates to true if T has a member function with signature:

```
int Read(gzFile gzf)
```

Required semantics

- Must populate *this with contents read from gzf
- Should return the number of decompressed bytes read on success, -1 on failure

9.7.2 HasGZWrite<T>

Evaluates to true if T has a member function with signature:

```
int Write(gzFile gzf) const
```

Required semantics

- Must write the contents of *this to gzf
- Should return the number of uncompressed bytes written on success, -1 on failure

9.7.3 Dwm::IsGZReadable<T>

9.7.4 Dwm::IsGZWritable<T>

9.8 HasSkipAnnotation<std::meta::info info>

Evaluates to true if the given reflection info has an annotation of the form [=Dwm::skip_io]

This requires C++26 reflection, specifically P2996 and P3394.

9.9 Streamed length

9.9.1 Constant streamed length

9.9.2 Constant streamed length via `static constexpr uint64_t StreamedLength()` member

9.9.3 Constant streamed length from C++26 reflection

When using C++26 reflection for serialization and deserialization of instances of a given type, we can determine at compile time whether or not the streamed length of the type is constant, and hence whether or not we can leverage `StreamIO` to provide `DescriptorIO`, `FileIO`, `BZ2IO`, `GZIO` and `ASIO` functionality automatically (via a pass through a `std::stringstream`).

Chapter 10

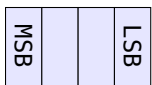
On-the-wire formats

10.1 On-the-wire formats, network byte order (MSB first)

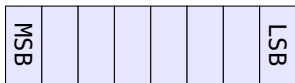
10.1.1 16 bit integral type (`int16_t`, `uint16_t`) encoding



10.1.2 32 bit integral type (`int32_t`, `uint32_t`) encoding

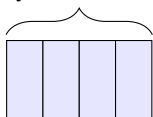


10.1.3 64 bit integral type (`int64_t`, `uint64_t`) encoding



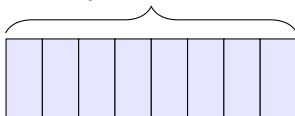
10.1.4 `float` encoding

4 bytes, RFC4506



10.1.5 `double` encoding

8 bytes, RFC4506



10.2 Encoded integral classes

10.2.1 Dwm::EncodedU64 encoding

`Dwm::EncodedU64` is encoded with an encoding byte followed by one to eight bytes representing an unsigned 64-bit integer value.

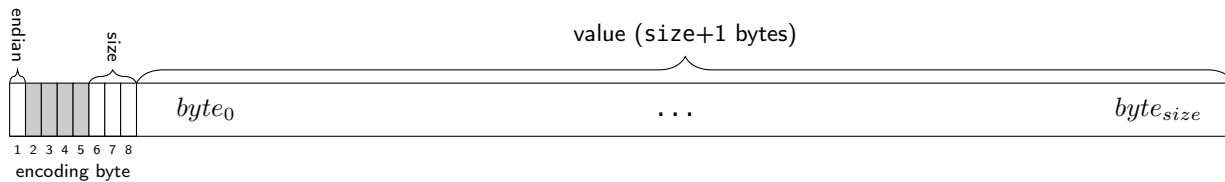


Figure 10.1: `Dwm::EncodedU64` encoding

Encoding byte

Four of the eight bits of the encoding byte are utilized. Table 10.1 shows the meaning of the endian bit.

endian	description
0	value is in little-endian format
1	value is in big-endian format

Table 10.1: Meaning of endian bit in `Dwm::EncodedU64` encoding byte

The size bits indicate the length of the value (in bytes) following the encoding byte, minus 1:

$$\text{length of value} = \text{size} + 1$$

Table 10.2 shows the translations between size and the length of the value.

size	0	1	2	3	4	5	6	7
length of value (bytes)	1	2	3	4	5	6	7	8

Table 10.2: meaning of size bits in `Dwm::EncodedU64` encoding byte

The remaining bits in the encoding byte are unused.

The value of a `Dwm::EncodedU64` is always written in native byte order (typically little-endian). The read member functions of `Dwm::EncodedU64` will convert endianness if needed.

Examples

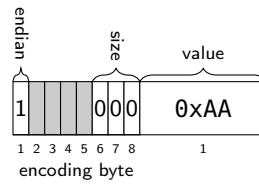


Figure 10.2: Dwm::EncodedU64 example: value 0xAA, big-endian.

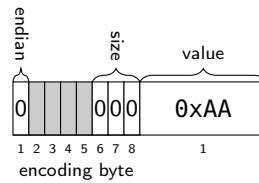


Figure 10.3: Dwm::EncodedU64 example: value 0xAA, little-endian.

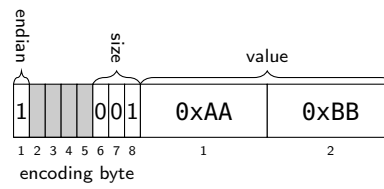


Figure 10.4: Dwm::EncodedU64 example: value 0xAABB, big-endian.

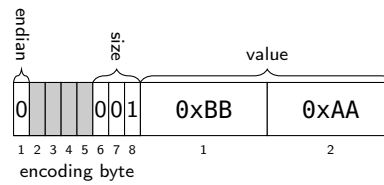


Figure 10.5: Dwm::EncodedU64 example: value 0xAABB, little-endian.

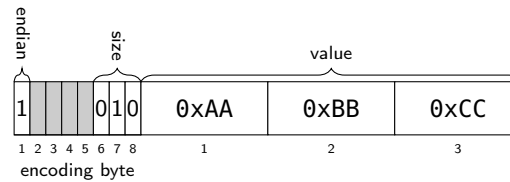


Figure 10.6: Dwm::EncodedU64 example: value 0xAABBCC, big-endian.

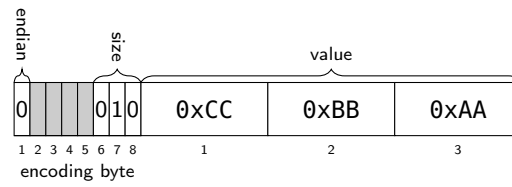


Figure 10.7: Dwm::EncodedU64 example: value 0xAABBCC, little-endian.

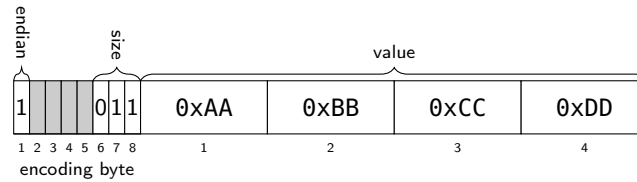


Figure 10.8: Dwm::EncodedU64 example: value 0xAABBCCDD, big-endian.

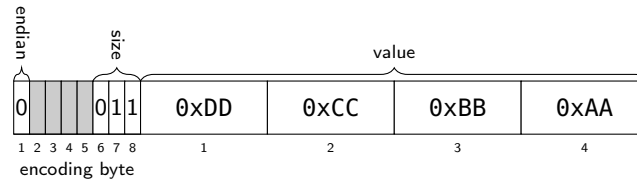


Figure 10.9: Dwm::EncodedU64 example: value 0xAABBCCDD, little-endian.

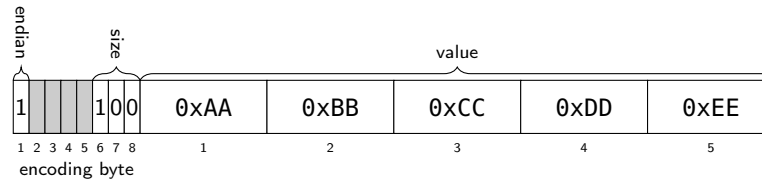


Figure 10.10: Dwm::EncodedU64 example: value 0xAABBCCDDEE, big-endian.



Figure 10.11: Dwm::EncodedU64 example: value 0xAABBCCDDEE, little-endian.

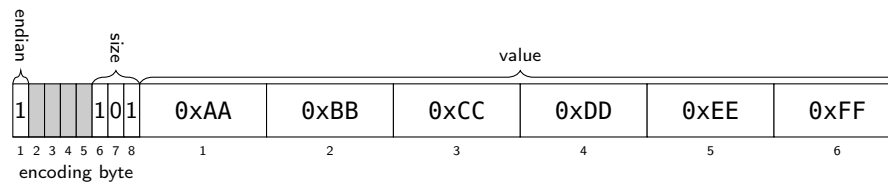


Figure 10.12: Dwm::EncodedU64 example: value 0xAABBCCDDEEFF, big-endian.

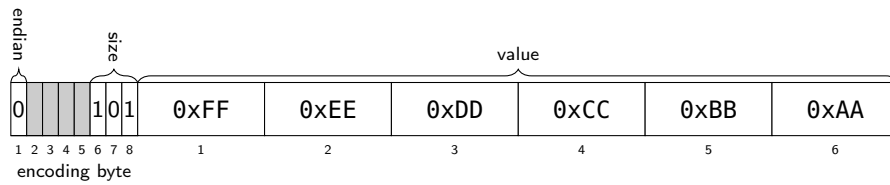


Figure 10.13: Dwm::EncodedU64 example: value 0xAABBCCDDEEFF, little-endian.

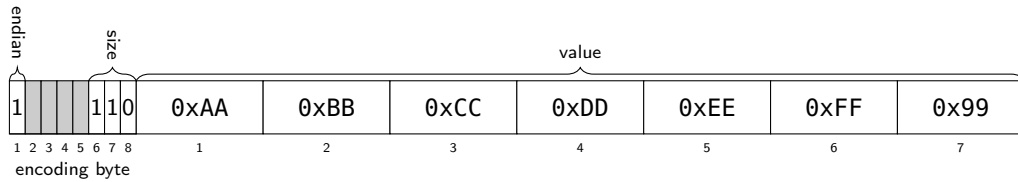


Figure 10.14: Dwm::EncodedU64 example: value 0xAABBCCDDEEFF99, big-endian.

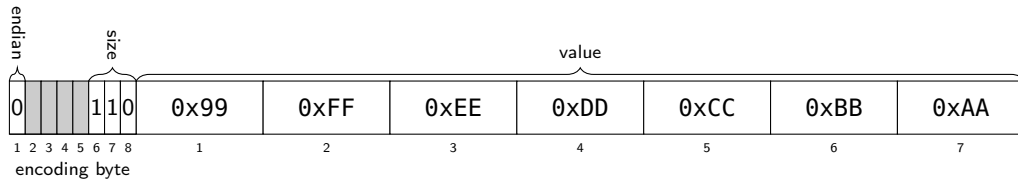


Figure 10.15: Dwm::EncodedU64 example: value 0xAABBCCDDEEFF99, little-endian.

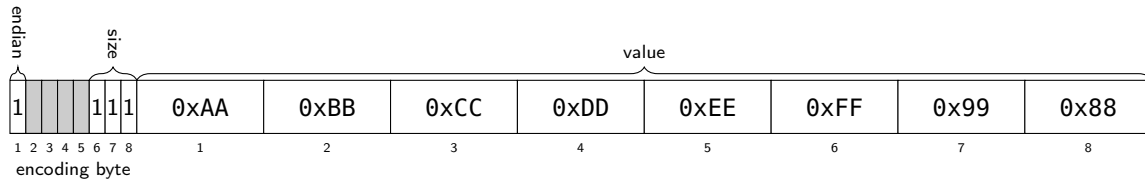


Figure 10.16: Dwm::EncodedU64 example: value 0xAABBCCDDEEFF9988, big-endian.

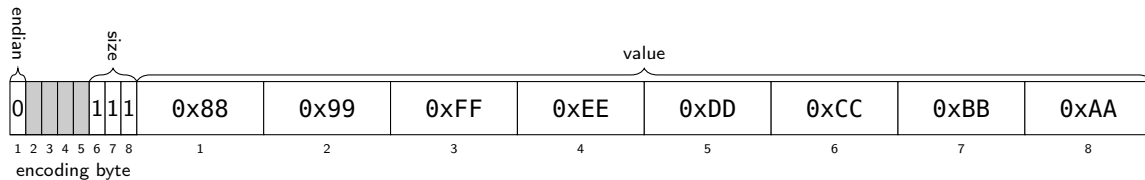
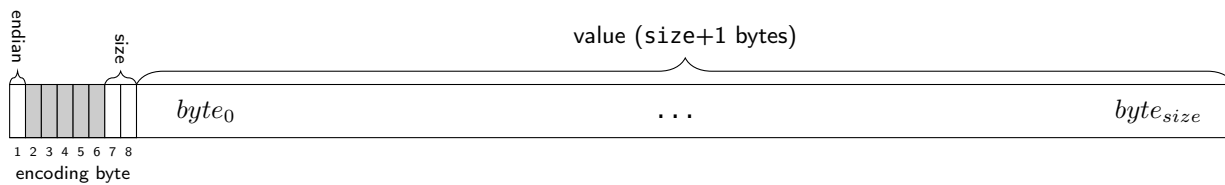


Figure 10.17: Dwm::EncodedU64 example: value 0xAABBCCDDEEFF9988, little-endian.

10.2.2 Dwm::EncodedU32 encoding

[Dwm::EncodedU32](#) is encoded with an encoding byte followed by one to four bytes representing an unsigned 32-bit integer value.

Figure 10.18: `Dwm::EncodedU32` encoding

Encoding byte

Three of the available eight bits of the encoding byte are utilized: one for endianness and two for size.

The meaning of the endian bit is shown in table 10.3.

endian	description
0	value is in little-endian format
1	value is in big-endian format

Table 10.3: Meaning of endian bit in `Dwm::EncodedU32` encoding byte

The size bits indicate the length of the value (in bytes) following the encoding byte, minus 1:

$$\text{length of value} = \text{size} + 1$$

Table 10.4 shows the translations between size and the length of the value.

size	0	1	2	3
length of value (bytes)	1	2	3	4

Table 10.4: meaning of size bits in `Dwm::EncodedU32` encoding byte

The value of a `Dwm::EncodedU32` is always written in native byte order (typically little-endian). The read member functions of `Dwm::EncodedU32` will convert endianness if needed.

Examples

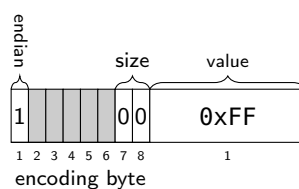


Figure 10.19: one byte big-endian value 255

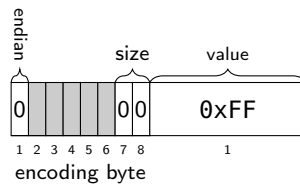


Figure 10.20: one byte little-endian value 255

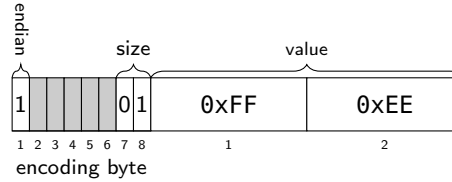


Figure 10.21: two byte big-endian value 65,518



Figure 10.22: three byte big-endian value 16,772,829

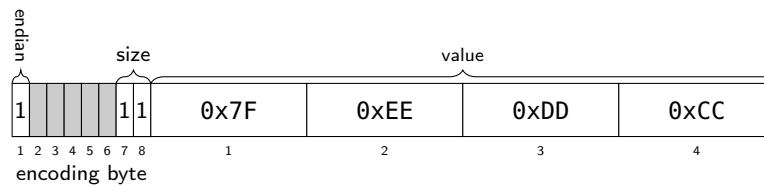


Figure 10.23: four byte big-endian value 86,554,432

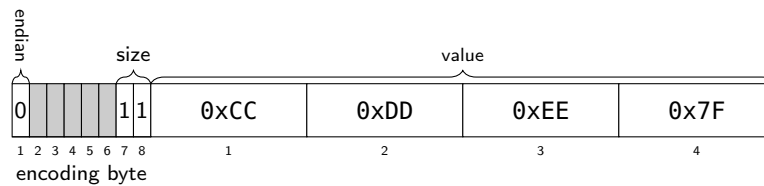
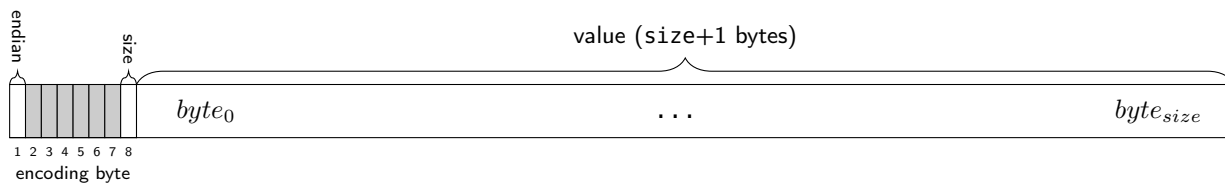


Figure 10.24: four byte little-endian value 86,554,432

10.2.3 Dwm::EncodedU16 encoding

`Dwm::EncodedU16` is encoded with an encoding byte followed by one or two bytes representing an unsigned 16-bit integer value.

Figure 10.25: `Dwm::EncodedU16` encoding

Encoding byte

Two of the eight bits of the encoding byte are utilized: one for endianness and one for size.

Table 10.5 shows the meaning of the endian bit.

endian	description
0	value is in little-endian format
1	value is in big-endian format

Table 10.5: Meaning of endian bit in `Dwm::EncodedU16` encoding byte

The size bit indicates the length of the value (in bytes) following the encoding byte, minus 1:

$$\text{length of value} = \text{size} + 1$$

Table 10.6 shows the translation between size and the length of the value.

size	0	1
length of value (bytes)	1	2

Table 10.6: meaning of size bits in encoding byte

The value of a `Dwm::EncodedU16` is always written in native byte order (typically little-endian). The read member functions of `Dwm::EncodedU16` will convert endianness if needed.

Examples

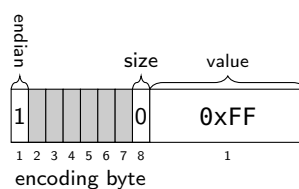


Figure 10.26: one byte big-endian value 255

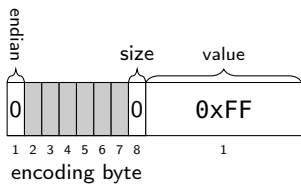


Figure 10.27: one byte little-endian value 255

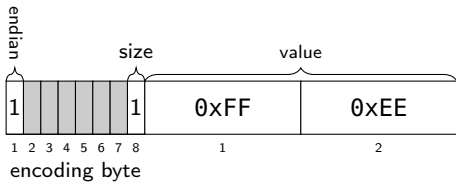


Figure 10.28: two byte big-endian value 65,518

10.2.4 Dwm::EncodedS64 encoding

`Dwm::EncodedS64` is encoded with an encoding byte followed by one to eight bytes representing a signed 64-bit integer value.

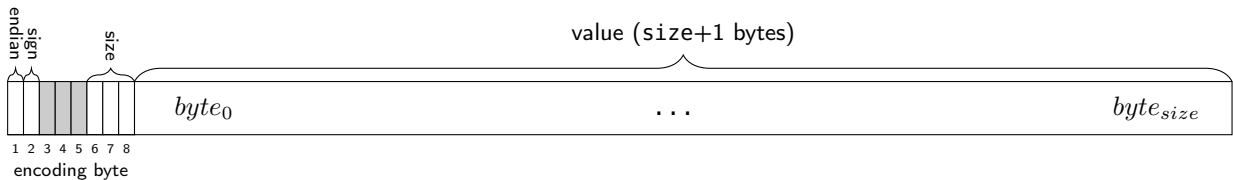


Figure 10.29: `Dwm::EncodedS64` encoding

Encoding byte

Five of the available bits in the encoding byte are utilized: one for endianness, one for sign and three to indicate the number of value bytes following the encoding byte.

Table 10.7 shows the meaning of the endian bit.

endian	description
0	value is in little-endian format
1	value is in big-endian format

Table 10.7: Meaning of endian bit in `Dwm::EncodedS64` encoding byte

Table 10.8 shows the meaning of the sign bit.

sign	description
0	value is positive
1	value is negative

Table 10.8: Meaning of sign bit in `Dwm::EncodedS64` encoding byte

The size bits indicate the length of the value (in bytes) following the encoding byte, minus 1:

length of value = size + 1

Table 10.9 shows the translations between size and the length of the value.

size	0	1	2	3	4	5	6	7
length of value (bytes)	1	2	3	4	5	6	7	8

Table 10.9: meaning of size bits in `Dwm::EncodedS64` encoding byte

The remaining bits in the encoding byte are unused.

The value of a `Dwm::EncodedS64` is always written in native byte order (typically little-endian). The read member functions of `Dwm::EncodedS64` will convert endianness if needed.

Examples

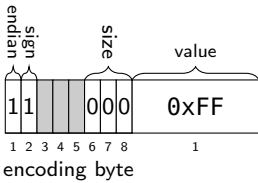


Figure 10.30: one byte big-endian value -255

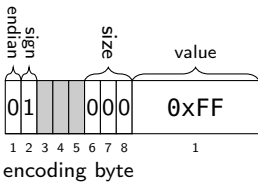


Figure 10.31: one byte little-endian value -255

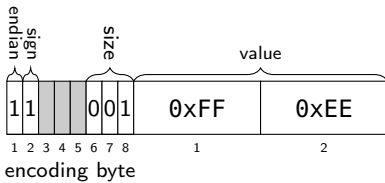


Figure 10.32: two byte big-endian value -65,518

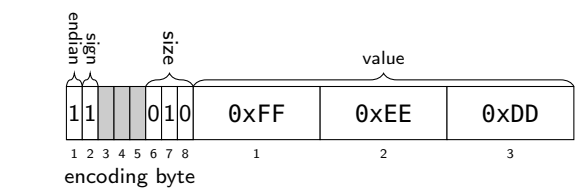


Figure 10.33: three byte big-endian value -16,772,829

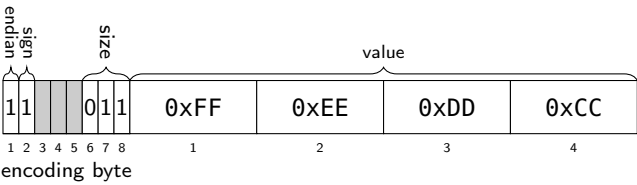


Figure 10.34: four byte big-endian value -4,293,844,428

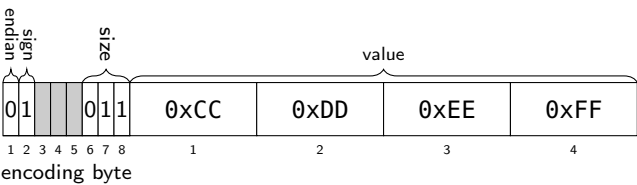


Figure 10.35: four byte little-endian value -4,293,844,428

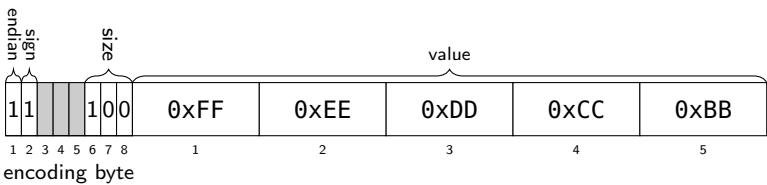


Figure 10.36: five byte big-endian value -1,099,224,173,755

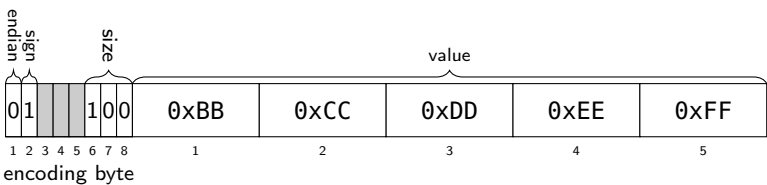


Figure 10.37: five byte little-endian value -1,099,224,173,755

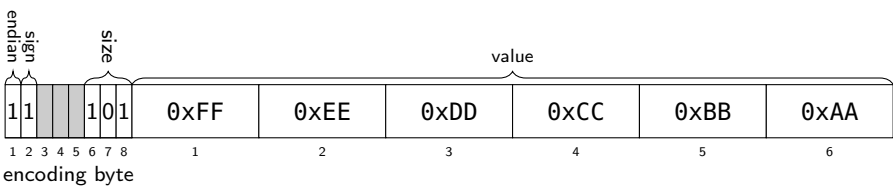


Figure 10.38: six byte big-endian value -281,401,388,481,450

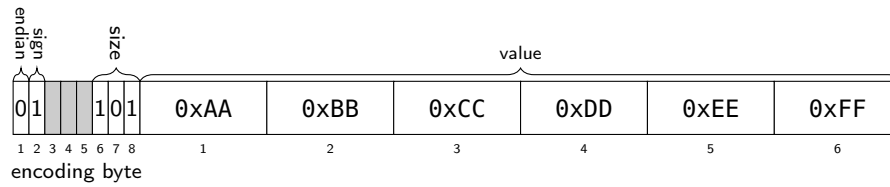


Figure 10.39: six byte little-endian value -281,401,388,481,450

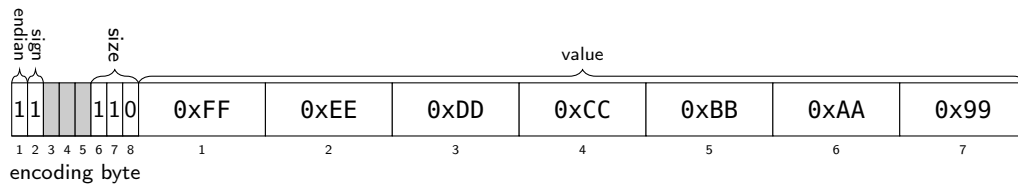


Figure 10.40: seven byte big-endian value -72,038,755,451,251,353

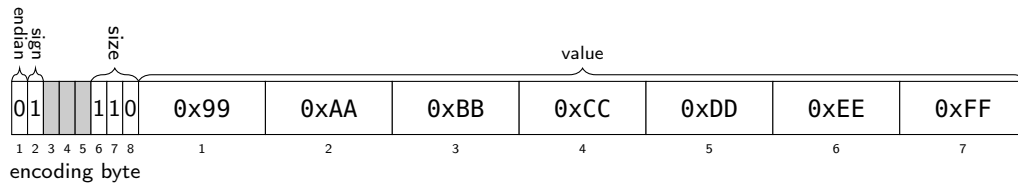


Figure 10.41: seven byte little-endian value -72,038,755,451,251,353

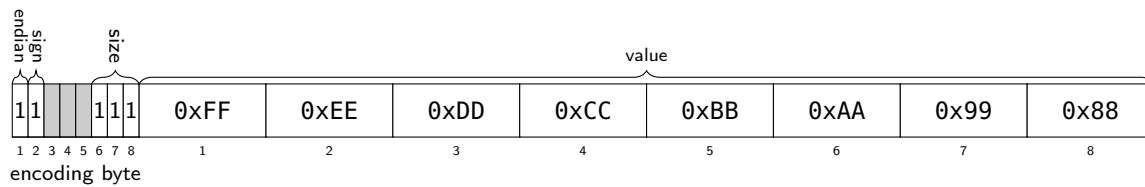


Figure 10.42: eight byte big-endian value -18,441,921,395,520,346,504

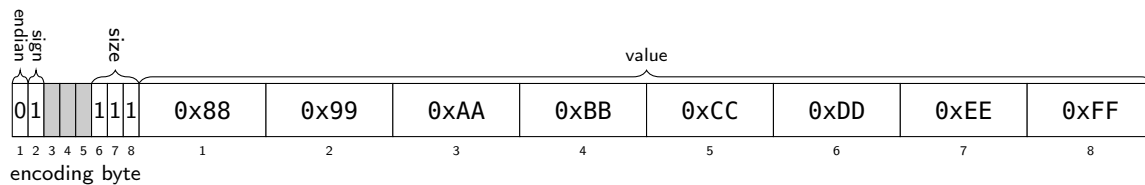
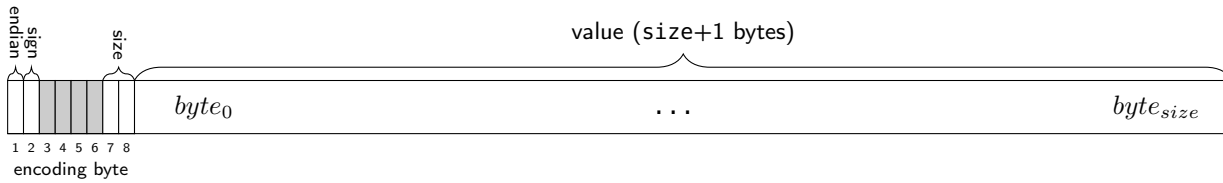


Figure 10.43: eight byte little-endian value -18,441,921,395,520,346,504

10.2.5 Dwm::EncodedS32 encoding

`Dwm::EncodedS32` is encoded with an encoding byte followed by one to four bytes representing a signed 32-bit integer value.

Figure 10.44: `Dwm::EncodedS32` encoding

Encoding byte

Four of the available bits in the encoding byte are utilized: one for endianness, one for sign and two to indicate the number of value bytes following the encoding byte.

Table 10.10 shows the meaning of the endian bit.

endian	description
0	value is in little-endian format
1	value is in big-endian format

Table 10.10: Meaning of endian bit in `Dwm::EncodedS32` encoding byte

Table 10.11 shows the meaning of the sign bit.

sign	description
0	value is positive
1	value is negative

Table 10.11: Meaning of sign bit in `Dwm::EncodedS32` encoding byte

The size bits indicate the length of the value (in bytes) following the encoding byte, minus 1:

$$\text{length of value} = \text{size} + 1$$

Table 10.12 shows the translations between size and the length of the value.

size	0	1	2	3
length of value (bytes)	1	2	3	4

Table 10.12: meaning of size bits in `Dwm::EncodedS32` encoding byte

The remaining bits in the encoding byte are unused.

The value of a `Dwm::EncodedS32` is always written in native byte order (typically little-endian). The read member functions of `Dwm::EncodedS32` will convert endianness if needed.

Examples

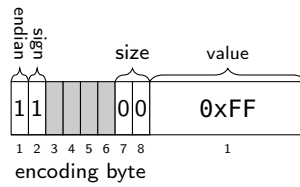


Figure 10.45: one byte big-endian value -255

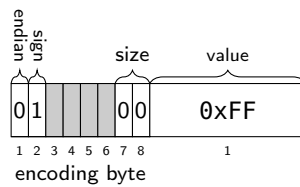


Figure 10.46: one byte little-endian value -255

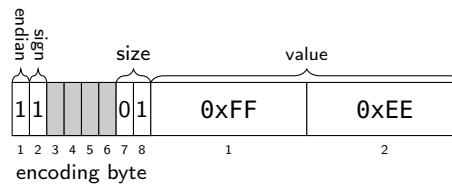


Figure 10.47: two byte big-endian value -65,518



Figure 10.48: three byte big-endian value -16,772,829

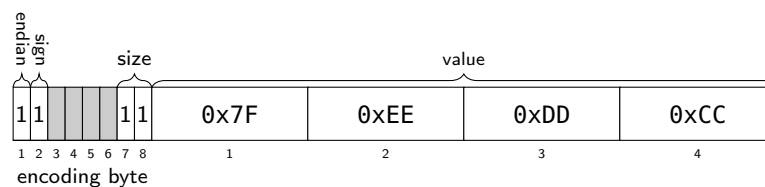


Figure 10.49: four byte big-endian value -86,554,432

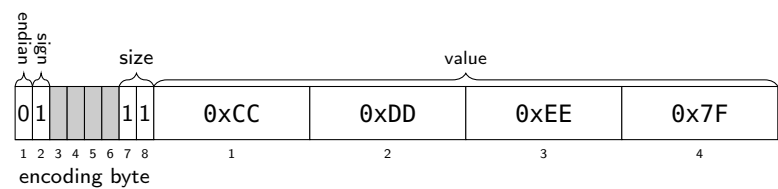


Figure 10.50: four byte little-endian value -86,554,432

10.2.6 Dwm::EncodedS16 encoding

Dwm::EncodedS16 is encoded with an encoding byte followed by one or two bytes representing a signed 16-bit integer value.

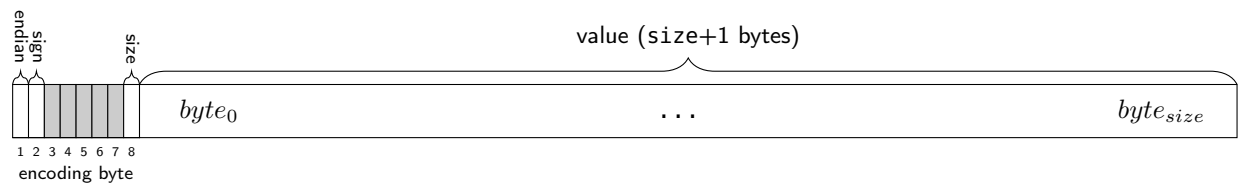


Figure 10.51: Dwm::EncodedS16 encoding

Encoding byte

Four of the available bits in the encoding byte are utilized: one for endianness, one for sign and two to indicate the number of value bytes following the encoding byte.

Table 10.13 shows the meaning of the endian bit.

endian	description
0	value is in little-endian format
1	value is in big-endian format

Table 10.13: Meaning of endian bit in Dwm::EncodedS16 encoding byte

Table 10.14 shows the meaning of the sign bit.

sign	description
0	value is positive
1	value is negative

Table 10.14: Meaning of sign bit in Dwm::EncodedS16 encoding byte

The size bits indicate the length of the value (in bytes) following the encoding byte, minus 1:

$length\ of\ value = size + 1$

Table 10.15 shows the translations between size and the length of the value.

	size	0	1
length of value (bytes)		1	2

Table 10.15: meaning of size bits in `Dwm::EncodedS16` encoding byte

The remaining bits in the encoding byte are unused.

The value of a `Dwm::EncodedS16` is always written in native byte order (typically little-endian). The read member functions of `Dwm::EncodedS16` will convert endianness if needed.

Examples

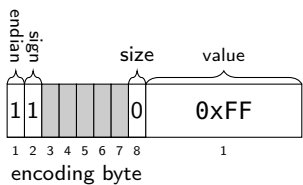


Figure 10.52: one byte big-endian value -255

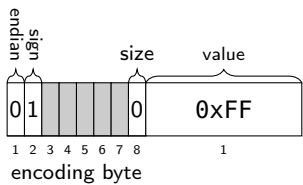


Figure 10.53: one byte little-endian value -255

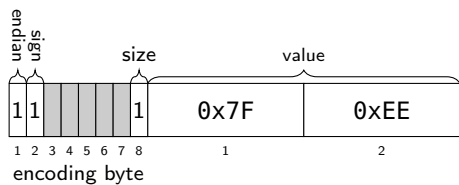


Figure 10.54: two byte big-endian value -32,750

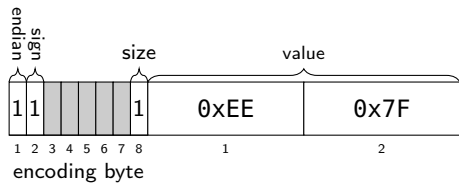


Figure 10.55: two byte little-endian value -32,750

10.3 Encoding of std classes and class templates

10.3.1 std::string encoding

std::string is encoded with a length field and then the characters in the string.

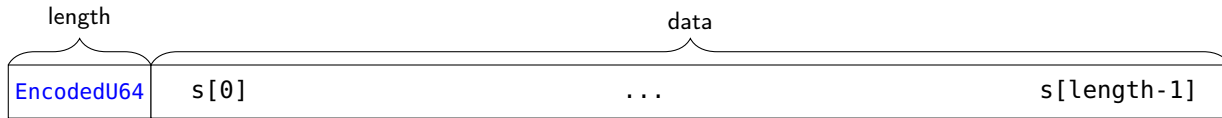


Figure 10.56: std::string encoding

10.3.2 std::optional<T> encoding

A std::optional<T> is encoded with a single byte to indicate whether or not the value is present, then the value if present.

10.3.3 std::string_view encoding

std::string is encoded with a length field and then the characters in the string_view data.

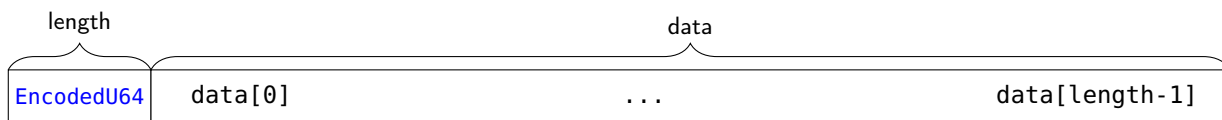


Figure 10.57: std::string_view encoding

NOTE: string_view is encoded in the same manner as string. We can't read a string_view from a source, we can only write it to a sink; it's const. But using the same encoding as string allows us to use Dwm::StreamIO to write data to a stringstream and then write a view of the stringstream to our sink. On the read side, we can read the data as a string, move the string into a istream and then read the data from the istream. This allows a library user to reduce the number of ::write() and ::read() system calls when using Dwm::DescriptorIO.

10.3.4 std::optional<T> encoding

A std::optional<T> is encoded with a single byte to indicate whether or not the value is present, then the value if present.

10.3.5 std::pair<FirstType,SecondType> encoding

The encoding of a std::pair is as you'd expect: the first value followed by the second value.

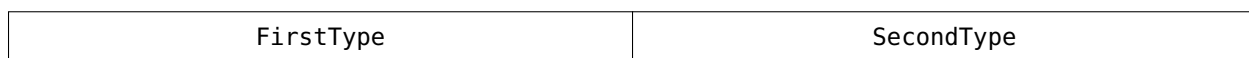


Figure 10.58: std::pair<FirstType,SecondType> encoding

10.3.6 `std::vector<T>` encoding

A `std::vector` is encoded with a 64-bit length field followed by `length` elements of the vector.

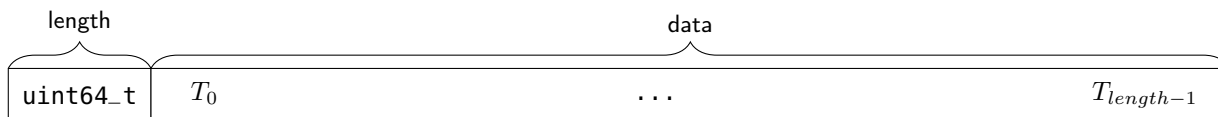


Figure 10.59: `std::vector<T>` encoding

NOTE: The length field is likely to be changed to use `Dwm::EncodedU64` encoding in the future.

10.3.7 `std::deque<T>` encoding

A `std::deque` is encoded with a 64-bit length field followed by `length` elements of the deque.

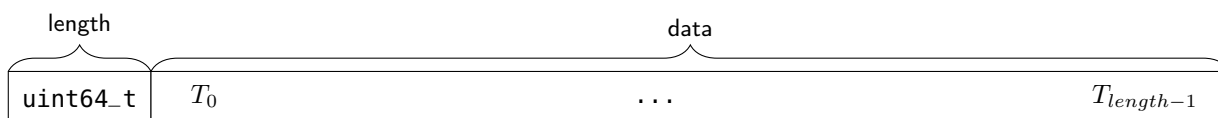


Figure 10.60: `std::deque<T>` encoding

NOTE: The length field is likely to be changed to use `Dwm::EncodedU64` in the future.

10.3.8 `std::list<T>` encoding

A `std::list` is encoded with a 64-bit length field followed by `length` elements of the list.

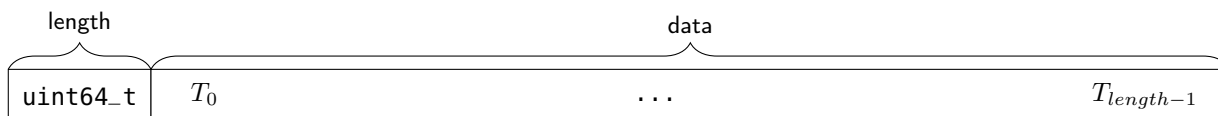


Figure 10.61: `std::list<T>` encoding

NOTE: The length field is likely to be changed to use `Dwm::EncodedU64` in the future.

10.3.9 `std::map<Key, Value>` encoding

A `std::map` is encoded with a 64-bit length field followed by `length` pairs of (Key, Value) elements of the map.

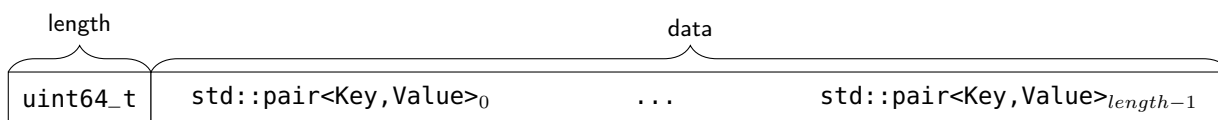


Figure 10.62: `std::map<Key, Value>` encoding

NOTE: The length field is likely to be changed to use `Dwm::EncodedU64` in the future.

10.3.10 `std::multimap<Key, Value>` encoding



Figure 10.63: `std::multimap<Key, Value>` encoding

NOTE: The length field is likely to be changed to use `Dwm::EncodedU64` in the future.

10.3.11 `std::unordered_map<Key, Value>` encoding

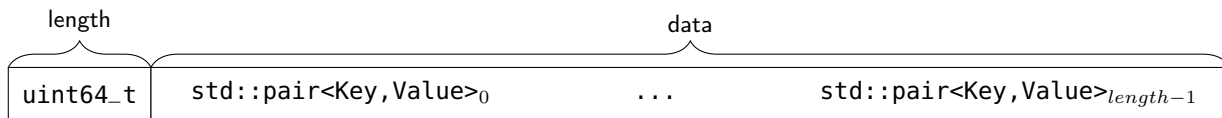


Figure 10.64: `std::unordered_map<Key, Value>` encoding

10.3.12 `std::unordered_multimap<Key, Value>` encoding

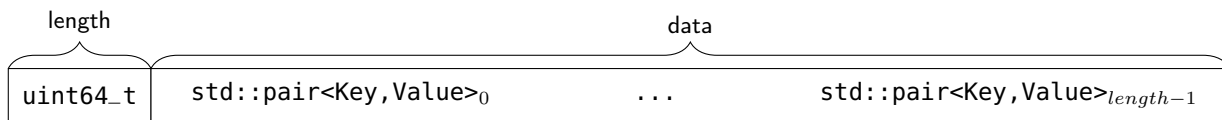


Figure 10.65: `std::unordered_multimap<Key, Value>` encoding

10.3.13 `std::set<T>` encoding

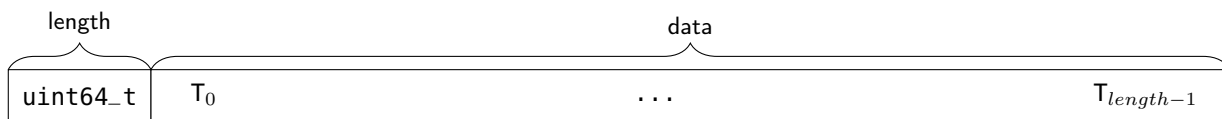


Figure 10.66: `std::set<T>` encoding

10.3.14 `std::multiset<T>` encoding

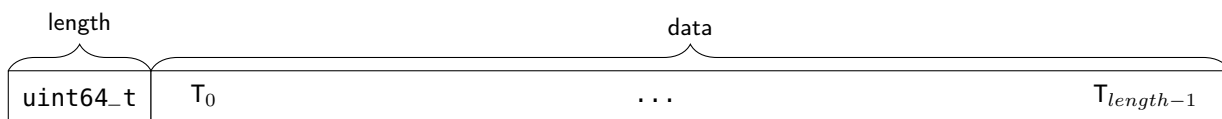
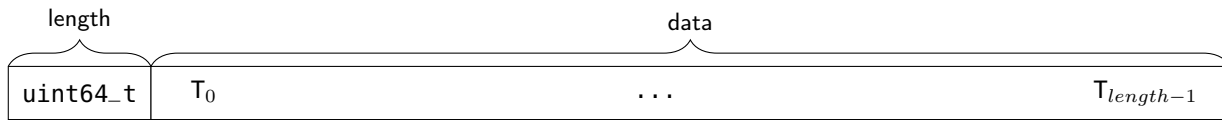
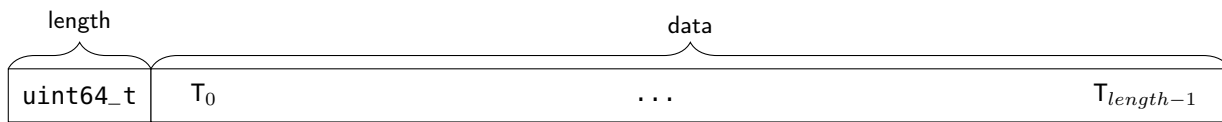


Figure 10.67: `std::multiset<T>` encoding

10.3.15 `std::unordered_set<T>` encodingFigure 10.68: `std::unordered_set<T>` encoding**10.3.16 `std::unordered_multiset<T>` encoding**Figure 10.69: `std::unordered_multiset<T>` encoding